

Simultaneous Localization And Mapping (SLAM)

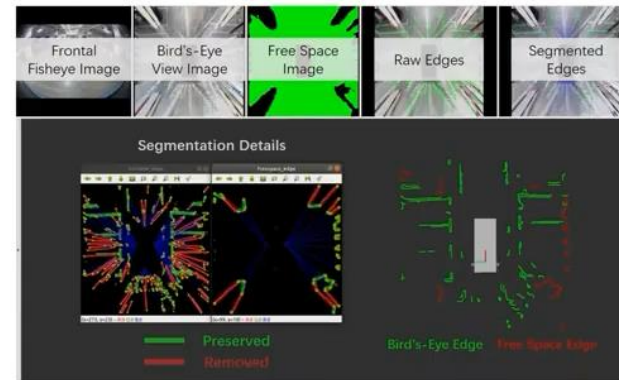


Credit: Outsight.ai

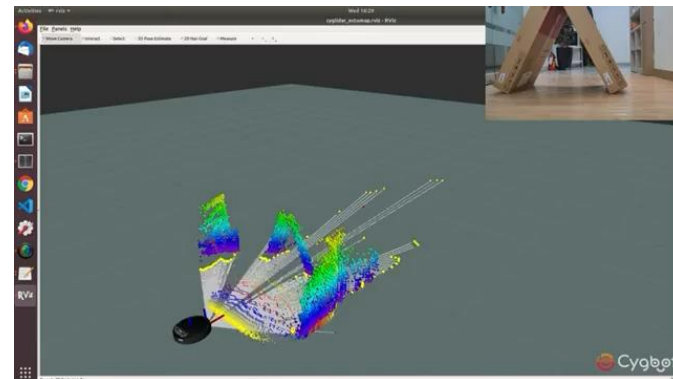


Credit: IV group, TU Delft

Video 1: Hybrid Edges Extraction (Polyline-based Segmentation)



Xiang, Z., et al. Hybrid bird's-eye edge based semantic visual SLAM for automated valet parking. ICRA 2021



Credit: Cygbot Lab

Overview

- What is SLAM?
- Classic Visual SLAM
- Deep Learning for Visual SLAM

What is Simultaneous Localization And Mapping (SLAM)?

What is Simultaneous **Localization** And **Mapping** (SLAM)?

Robot localization

Robot – a device, that moves through the environment.
E.g. an autonomous vehicle

Localization – estimating the robot's pose

Why not GNSS positioning?

Maddern, W., Pascoe, G., Linegar, C. and Newman, P.. 1 year, 1000 km: The Oxford RobotCar dataset. IJRR, 2017



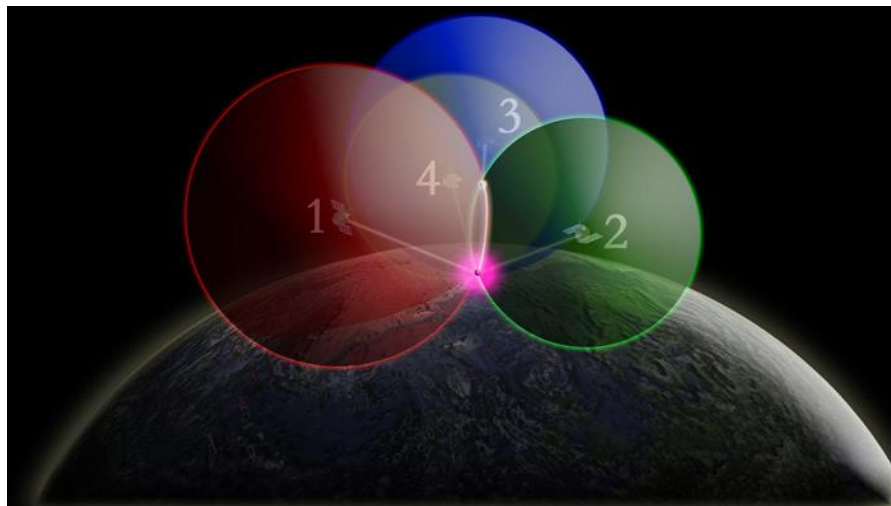
Green: RTK. Red: GPS

Note that, the selected trajectories are not labeled with “poor GPS” in Oxford RobotCar dataset

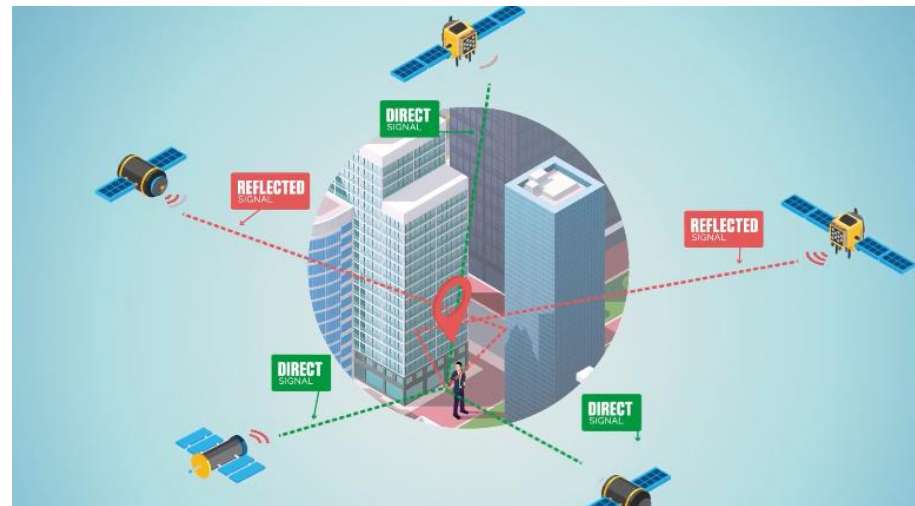


Global Navigation Satellite System (GNSS) includes Galileo (EU), GPS (USA), GLONASS (Russia), BeiDou (China)

Why not GNSS positioning?



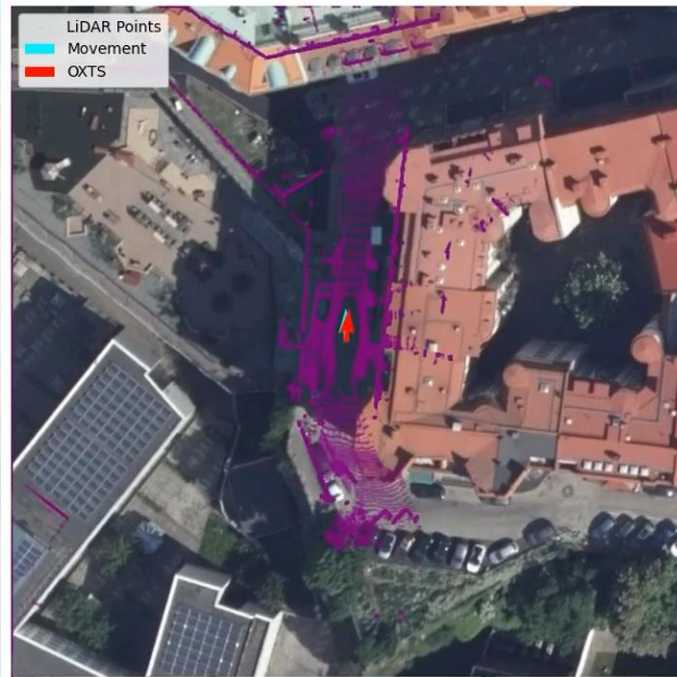
https://oceanservice.noaa.gov/education/tutorial_geodesy/geo09_gps.html



Credit: EU Agency for the Space Programme

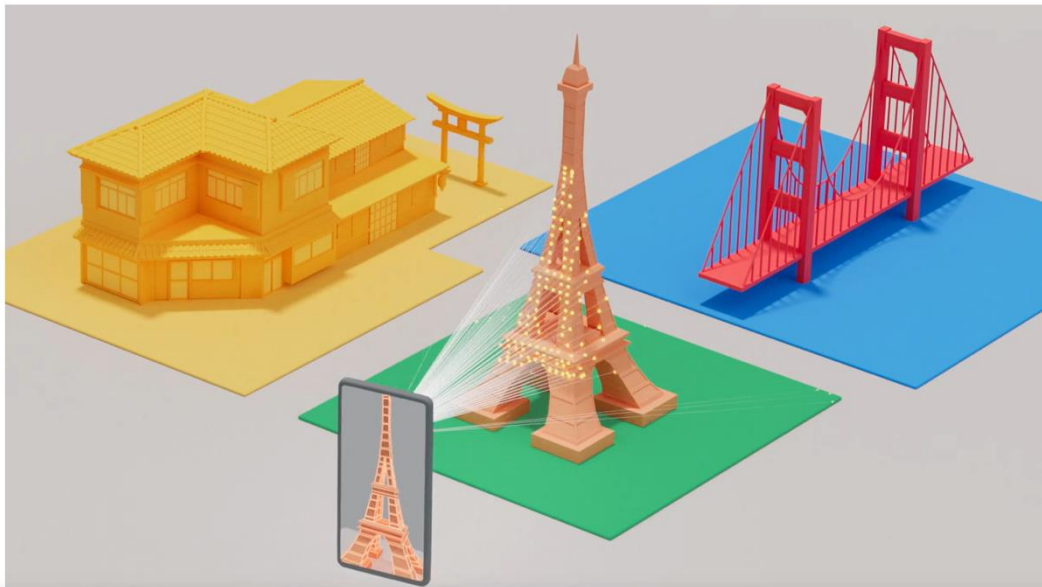
- GNSS suffers from the **multi-path effect**
- Not reliable near tall buildings or under trees

Why not GNSS positioning?



Alibeigi, M., et al. Zenseact open dataset: A large-scale and diverse multimodal dataset for autonomous driving. ICCV, 2023.

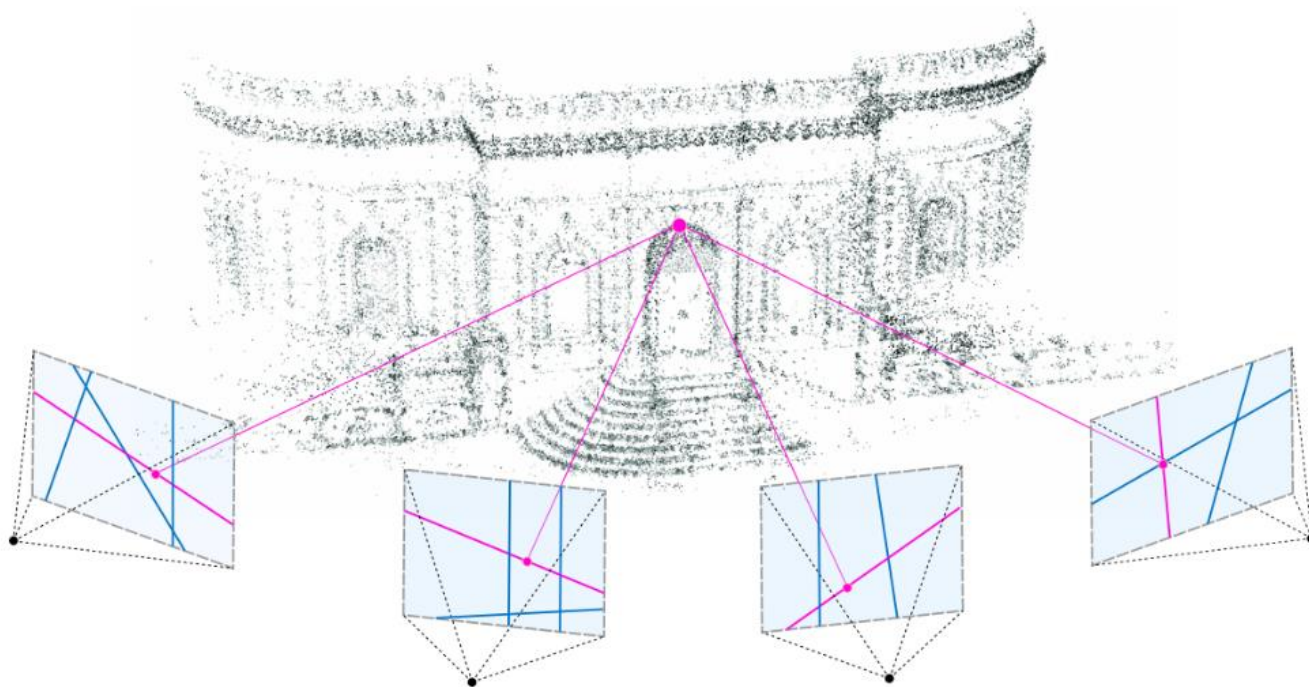
Map-based localization



Credit: Google's Visual Positioning System

Compare your surroundings to a pre-constructed map to determine your location

Robot mapping



To construct a model of the environment, it is necessary to know the robot's pose

Localization and mapping represent a “chicken or the egg” problem

- Localizing of a robot with respect to a map requires an estimate of the map
- Mapping an environment from data acquired by a robot requires an estimate of the robot localization

What is Simultaneous Localization And Mapping (SLAM)?

What is Simultaneous Localization And Mapping (SLAM)?

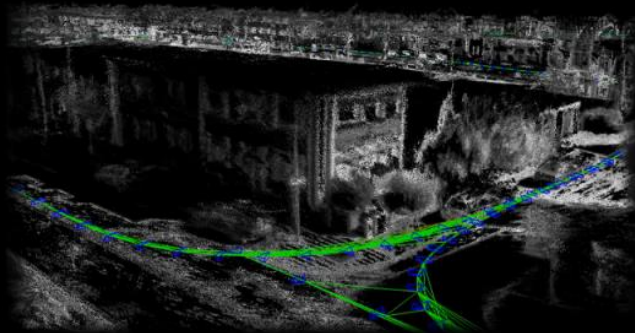
Constructing or updating a map of an unknown environment while simultaneously keeping track of a robot's pose within it

Any questions?

Classic Visual SLAM

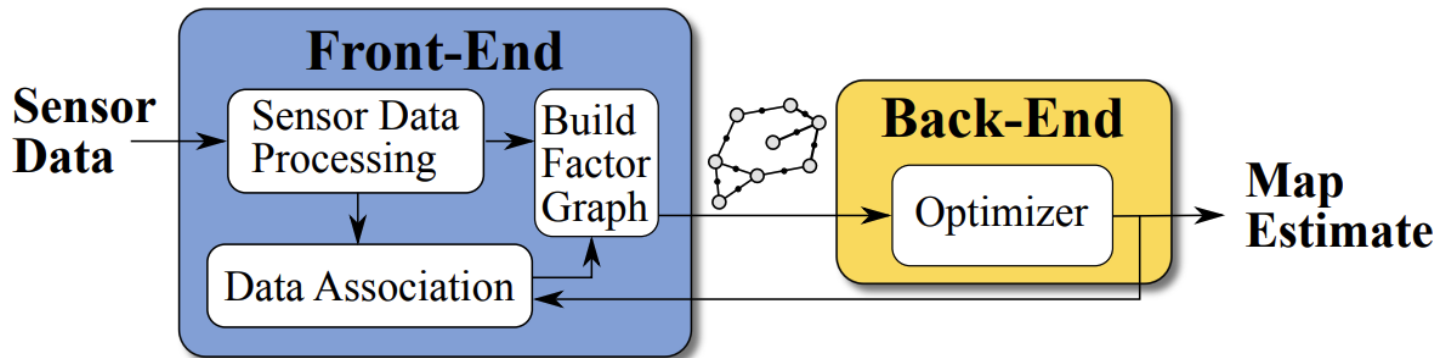
Large-Scale Direct SLAM with Stereo Cameras

Jakob Engel, Jörg Stückler, Daniel Cremers
IROS 2015, Hamburg

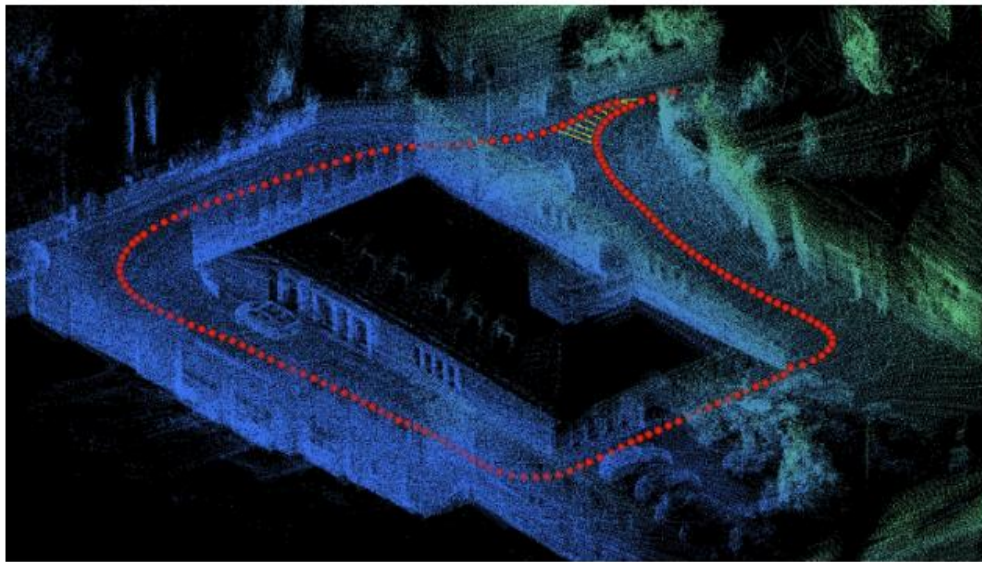


Computer Vision Group
Technical University Munich





- SLAM consists of a front-end and a back-end
- Front-end -> Construct a pose graph
- Back-end -> Optimize the pose graph and the map

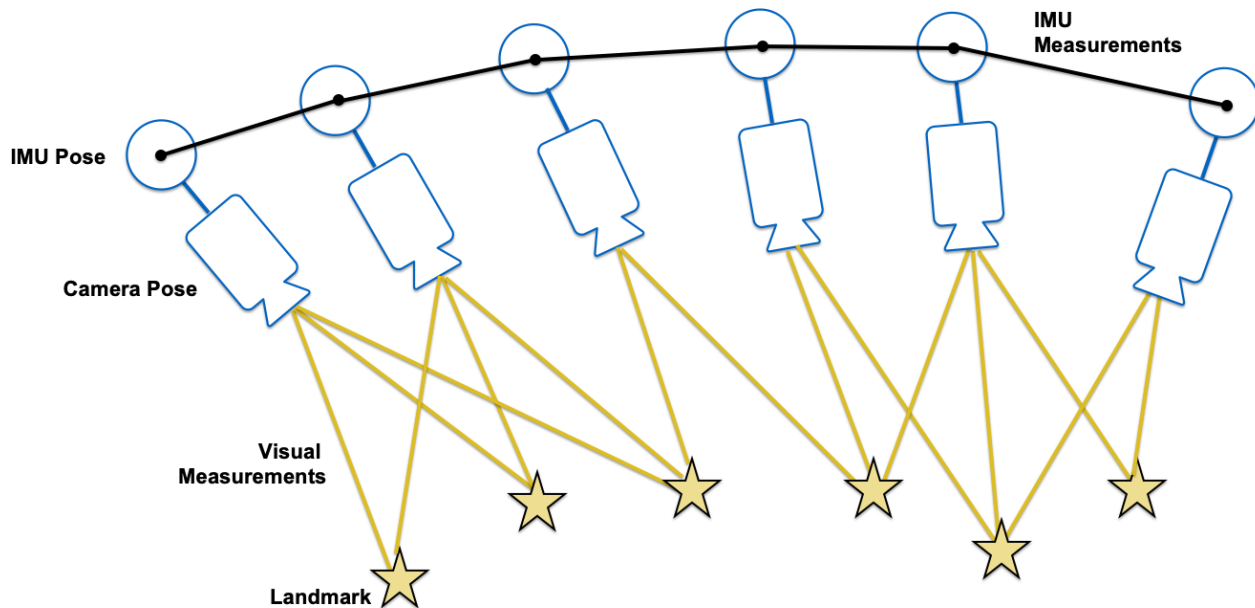


The front-end is responsible for

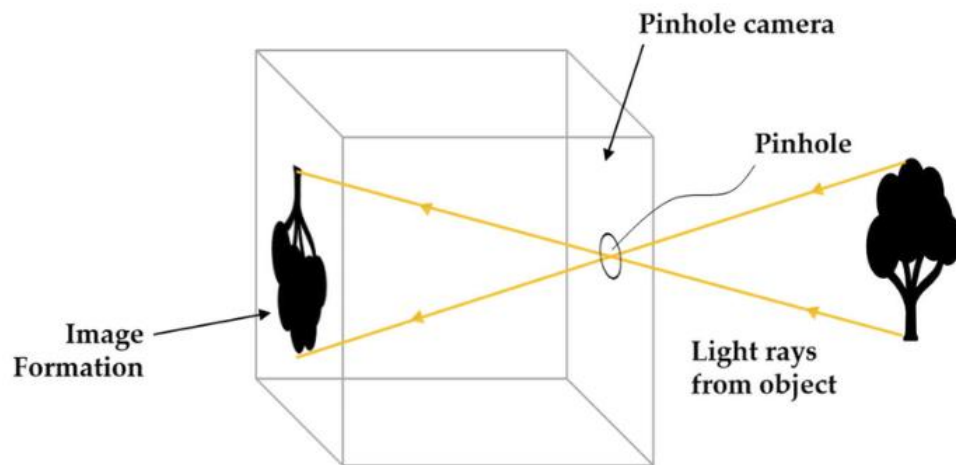
- Feature extraction, matching, and outlier removal
- Loop closure detection

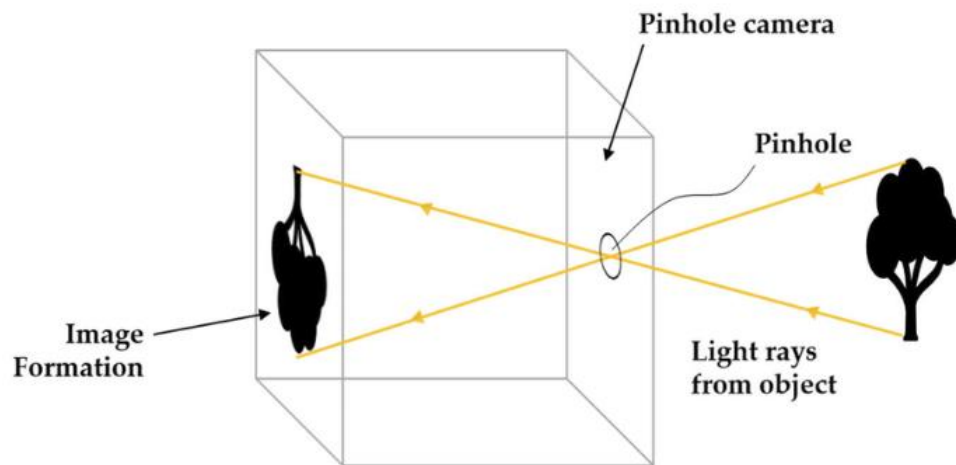
The back-end is responsible for

- Pose and structure optimization

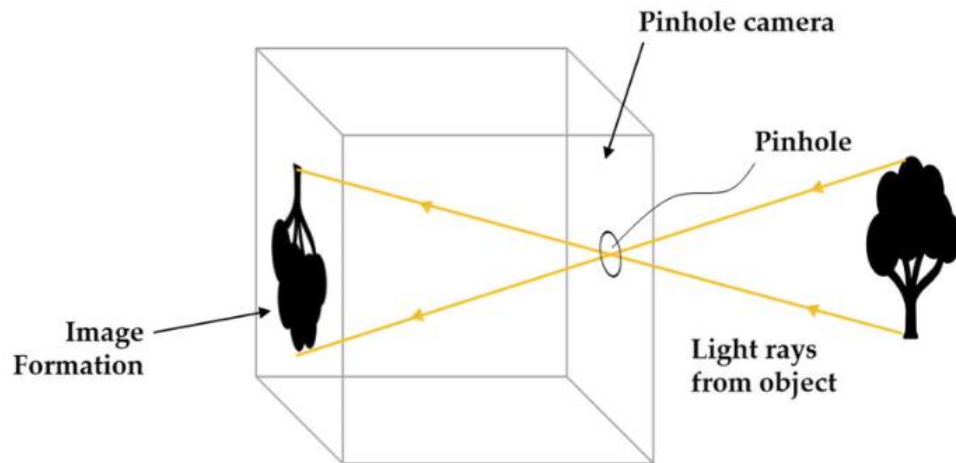


Given a sequence of images, estimate the motion

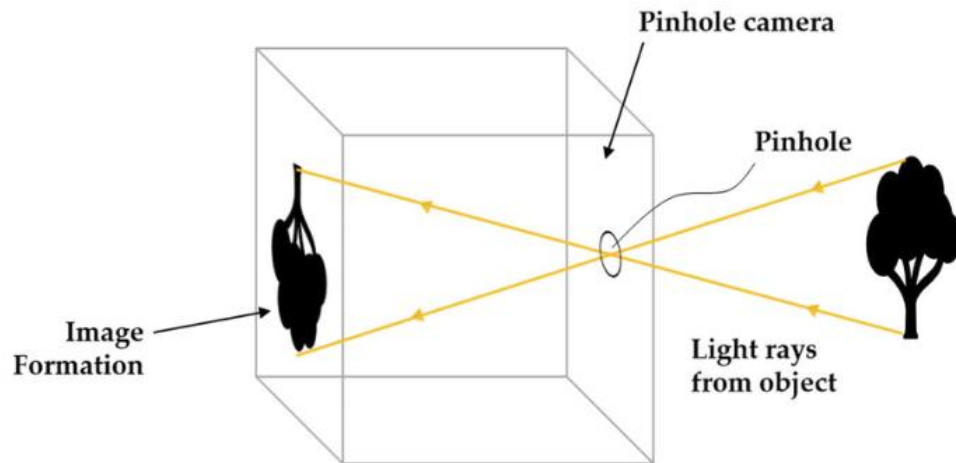




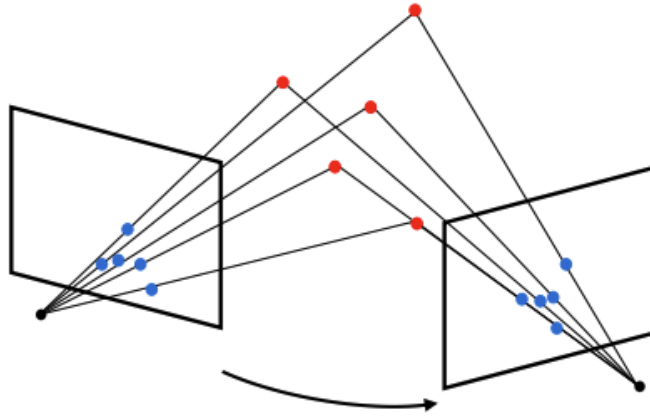
- Rays of light enters the camera through an infinitesimally small aperture

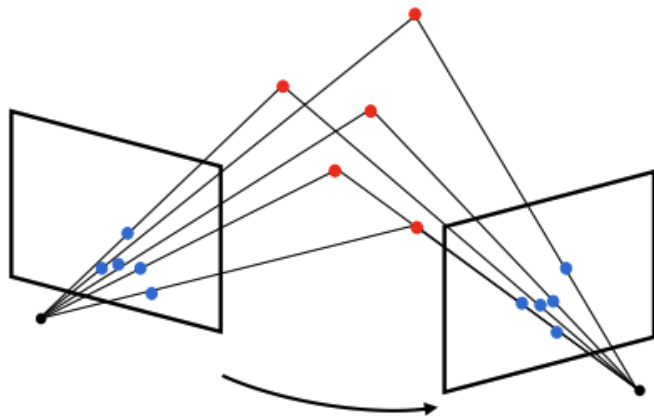


- Rays of light enters the camera through an infinitesimally small aperture
- The intersection of the light rays with the image plane forms the image of the object

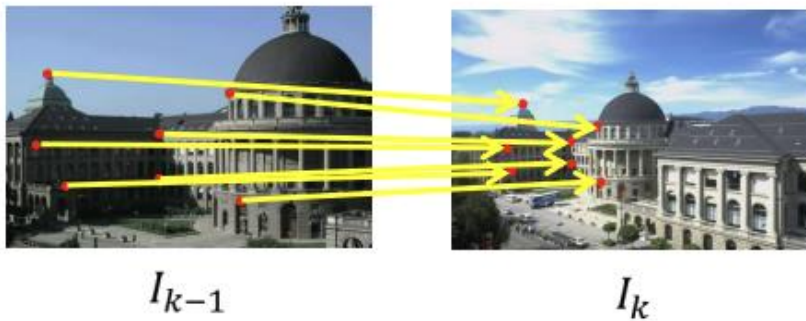


- Rays of light enters the camera through an infinitesimally small aperture
- The intersection of the light rays with the image plane forms the image of the object
- Such a mapping from three dimensions onto two dimensions is called **perspective projection**



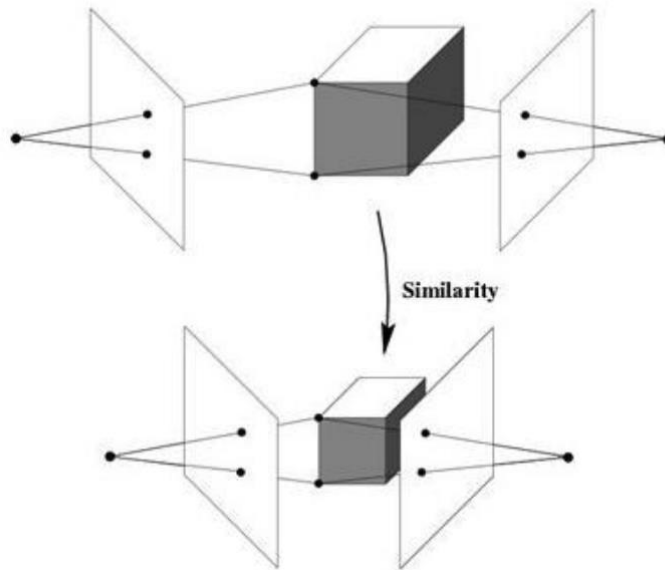


Source: http://mrsl.grasp.upenn.edu/loianog/tutorial_ICRA2016/VO_Tutorial.pdf



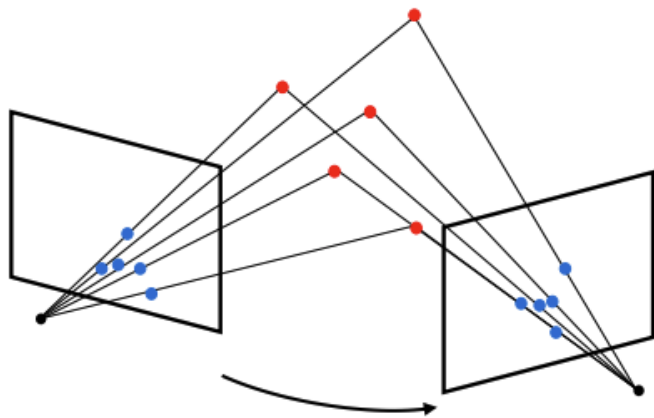
- Find pixel correspondences between two frames
- Need at least 5 corresponding points

Source: https://rpg.ifi.uzh.ch/docs/teaching/2017/08_multiple_view_geometry_2.pdf



If we scale the entire scene with a factor, the projected pixels remain at the same location

Why do we need at least 5 points?



Unknowns:

- The motion up to a scale (3 rotation, 2 translation)
- Coordinates of the n 3D points ($3n$)

Knowns:

- n correspondences ($4n$)

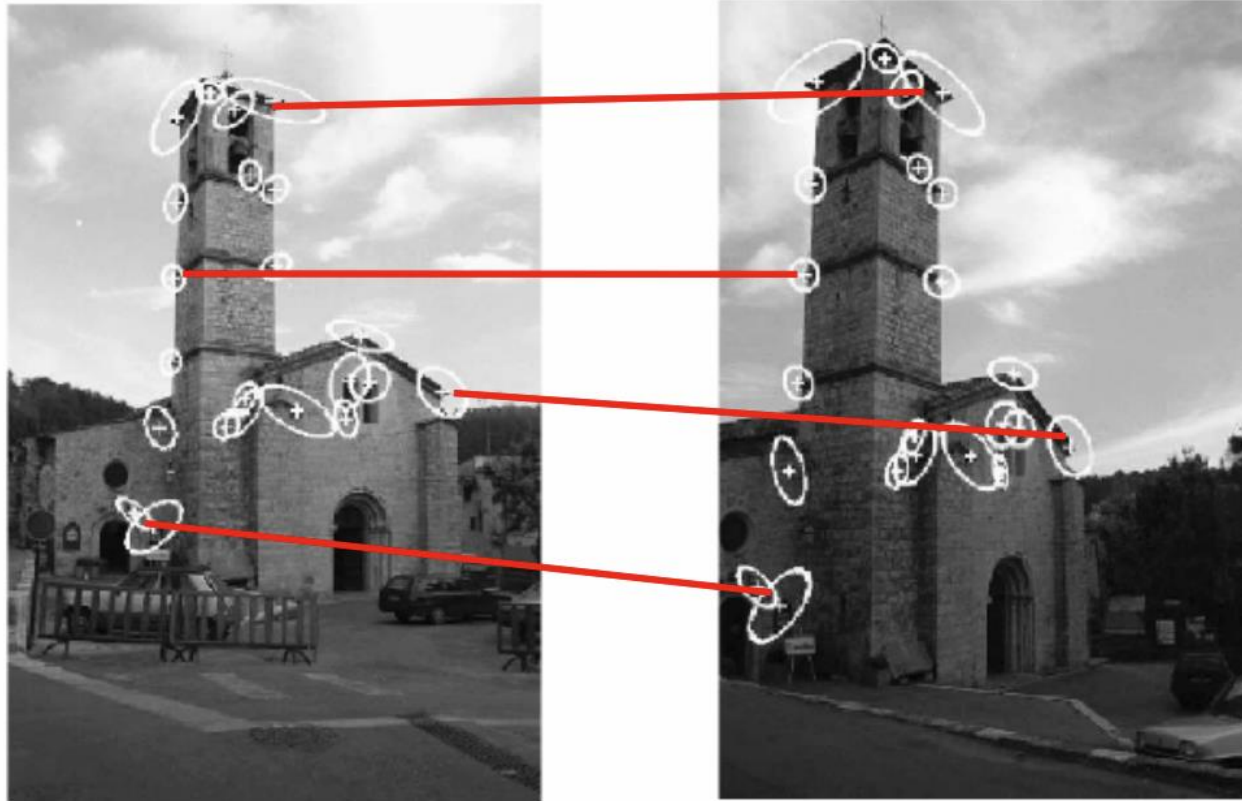
$$4n \geq 5 + 3n$$

$$n \geq 5$$

To resolve the scale:

- Use stereo camera
- Measure the distance travelled between first two frames (IMU)
- Use depth sensing sensors (depth camera, LiDAR)

EPFL Correspondences matching



Basic assumptions:

- Most scene points are visible in both images
- Corresponding image regions are similar

Basic assumptions:

- Most scene points are visible in both images
- Corresponding image regions are similar

These assumptions hold if:

- The distance of points from the cameras is much larger than the distance between cameras

The Correspondence Problem is a “search” problem

- Given an element in the left image, search for the corresponding element in the right image

The Correspondence Problem is a “search” problem

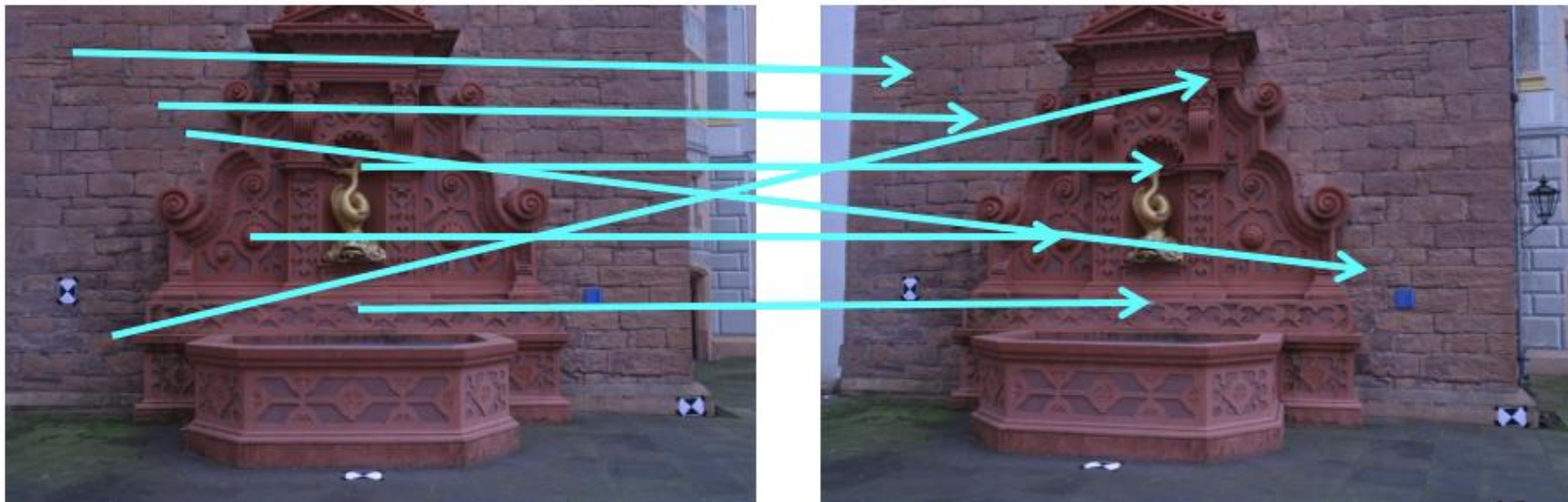
- Given an element in the left image, search for the corresponding element in the right image

We must choose:

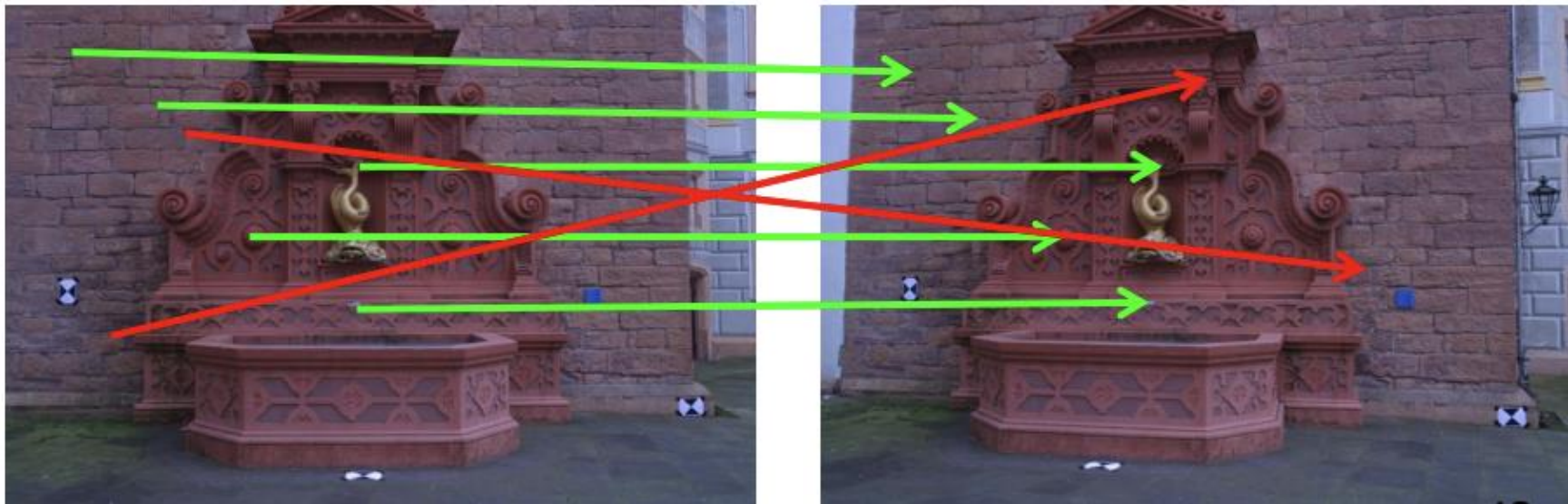
- Elements to match
- A similarity measure to compare elements

Two classes of algorithms:

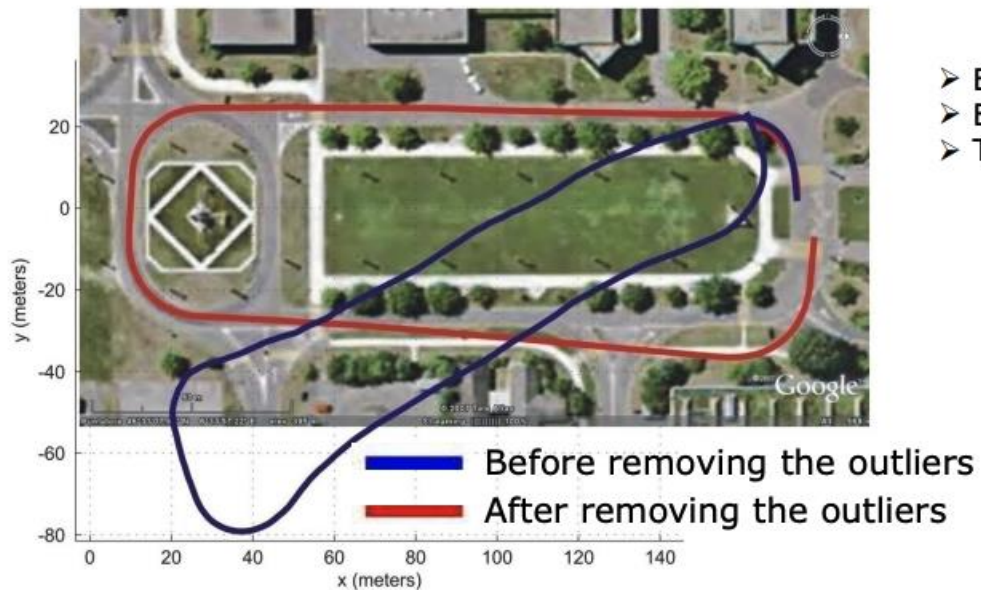
- Correlation-based algorithms: Produce a **dense** set of correspondences
- Feature-based algorithms: Produce a **sparse** set of correspondences



In practice, we establish many correspondences



There are often outliers



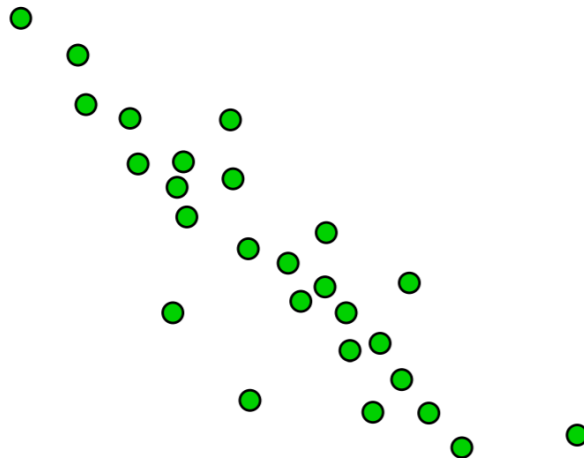
- Error at the loop closure: 6.5 m
- Error in orientation: 5 deg
- Trajectory length: 400 m

Fishler & Bolles, 1981

RANdom SAmple Consensus, RANSAC

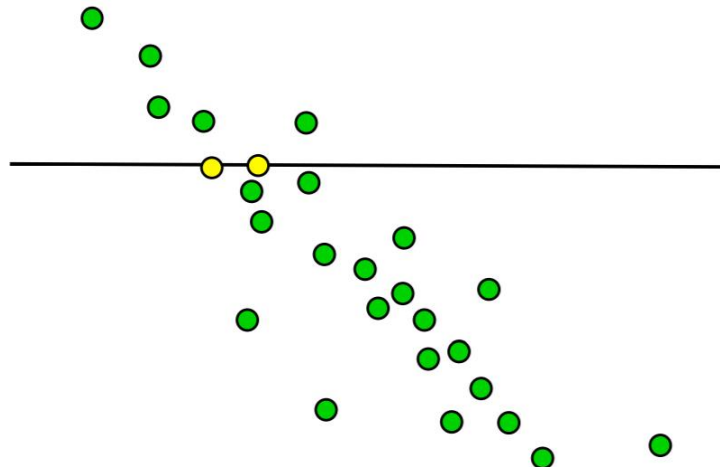
- standard method for model fitting in the presence of outliers

We want to fit a line to data points



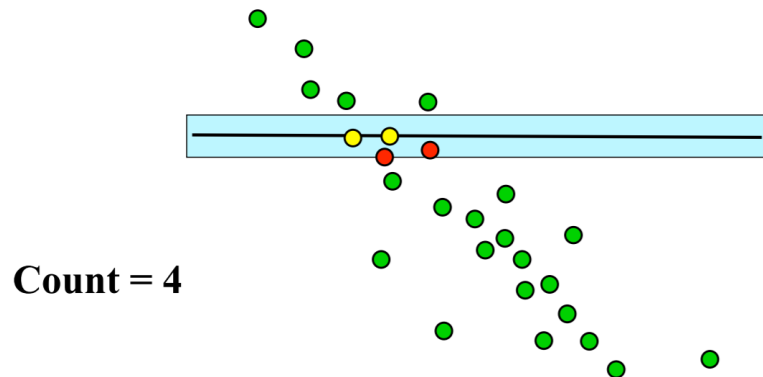
We want to fit a line to data points

- Every 2 points can define a line



We want to fit a line to data points

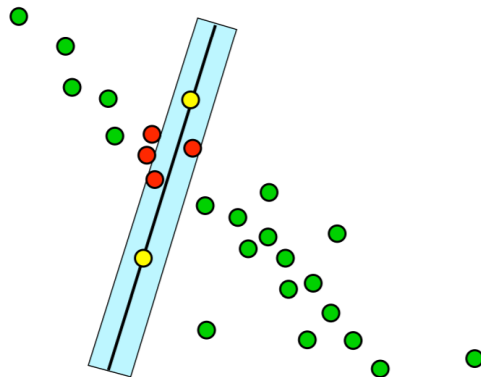
- Every 2 points can define a line
- Randomly select 2 points, fit a model, count inliers



We want to fit a line to data points

- Every 2 points can define a line
- Randomly select 2 points, fit a model, count inliers
- Iterate over all data points

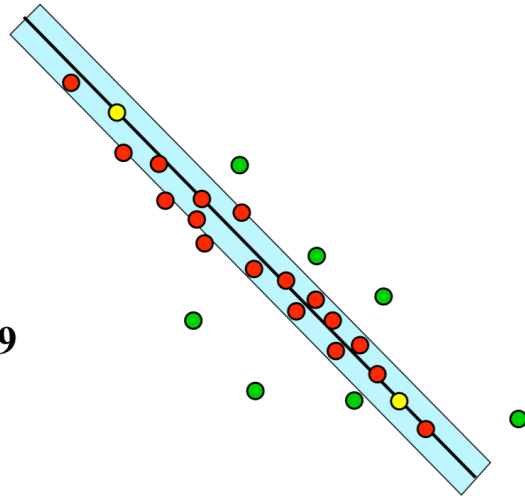
Count = 6



We want to fit a line to data points

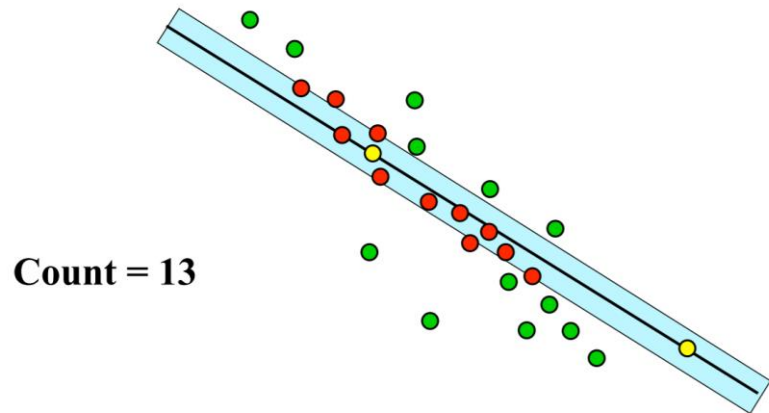
- Every 2 points can define a line
- Randomly select 2 points, fit a model, count inliers
- Iterate over all data points

Count = 19



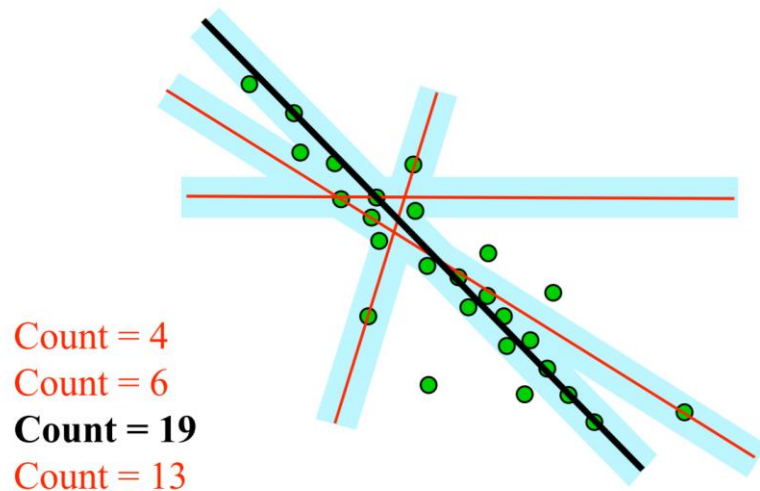
We want to fit a line to data points

- Every 2 points can define a line
- Randomly select 2 points, fit a model, count inliers
- Iterate over all data points



We want to fit a line to data points

- Every 2 points can define a line
- Randomly select 2 points, fit a model, count inliers
- Iterate over all data points
- Pick the model with most inliers
- Refit the line use all inliers



RANdom SAmple Consensus, RANSAC

- Iterative method for outlier detection
- Non-deterministic with user defined probability of success

RANdom SAmple Consensus, RANSAC

- Iterative method for outlier detection
- Non-deterministic with user defined probability of success

Output:

- Best fit of model
- Inliers: belonging to the model
- Outliers: not belonging to the model

RANSAC applied to motion estimation:

- Randomly select 5 correspondences to estimate the motion

RANSAC applied to motion estimation:

- Randomly select 5 correspondences to estimate the motion
- Use the estimated motion to reproject the points in one view to another

RANSAC applied to motion estimation:

- Randomly select 5 correspondences to estimate the motion
- Use the estimated motion to reproject the points in one view to another
- Count the number of inliers given a defined threshold, e.g. distance in pixels

RANSAC applied to motion estimation:

- Randomly select 5 correspondences to estimate the motion
- Use the estimated motion to reproject the points in one view to another
- Count the number of inliers given a defined threshold, e.g. distance in pixels
- Iterate this process

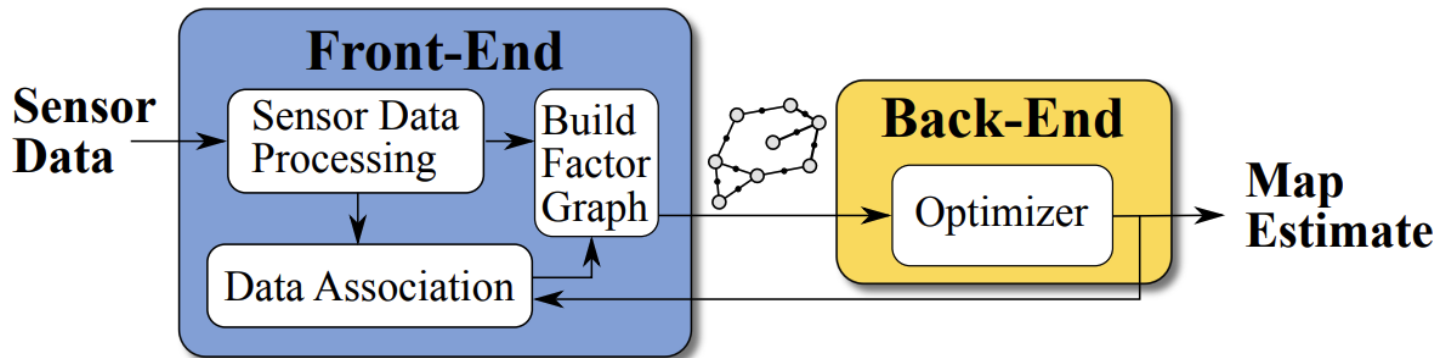
RANSAC applied to motion estimation:

- Randomly select 5 correspondences to estimate the motion
- Use the estimated motion to reproject the points in one view to another
- Count the number of inliers given a defined threshold, e.g. distance in pixels
- Iterate this process
- Pick the model with most inliers

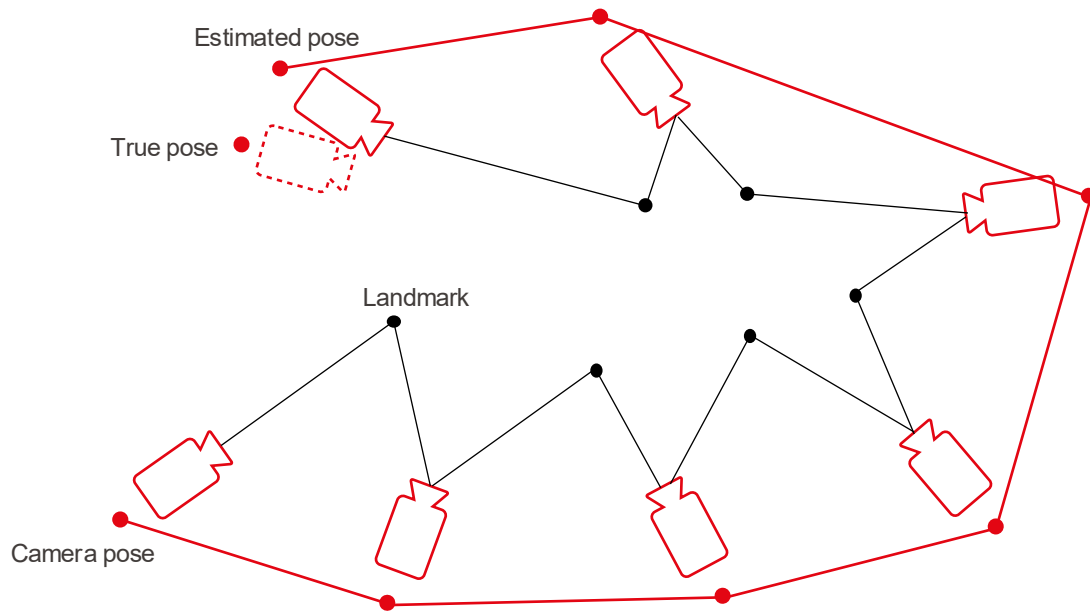
RANSAC applied to motion estimation:

- Randomly select 5 correspondences to estimate the motion
- Use the estimated motion to reproject the points in one view to another
- Count the number of inliers given a defined threshold, e.g. distance in pixels
- Iterate this process
- Pick the model with most inliers
- Re-estimate the motion use all inliers

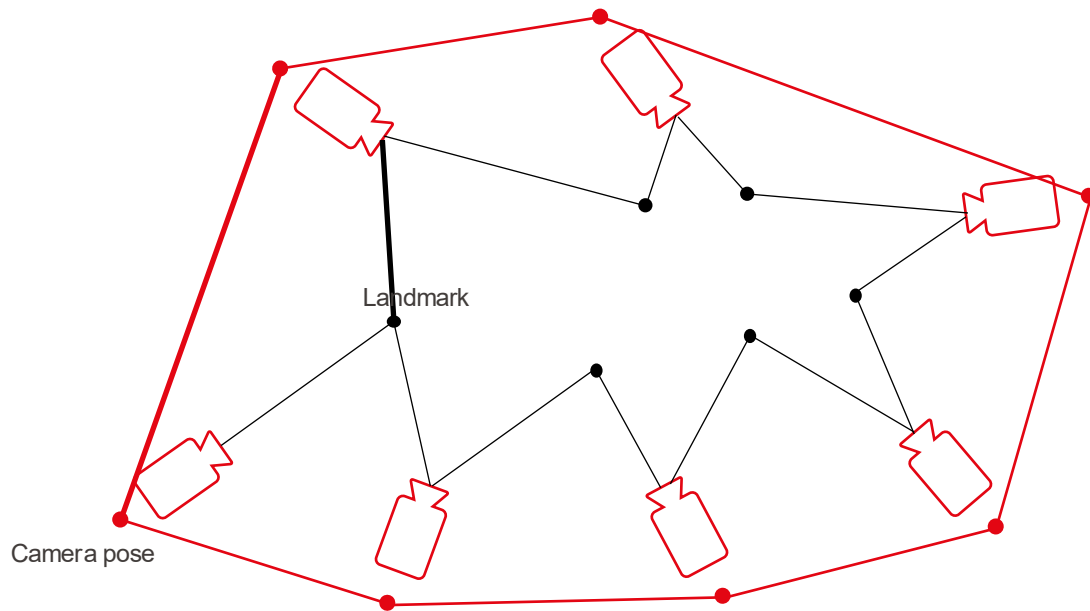
Any questions?



- Front-end -> Construct a pose graph



Visual odometry suffers from drift

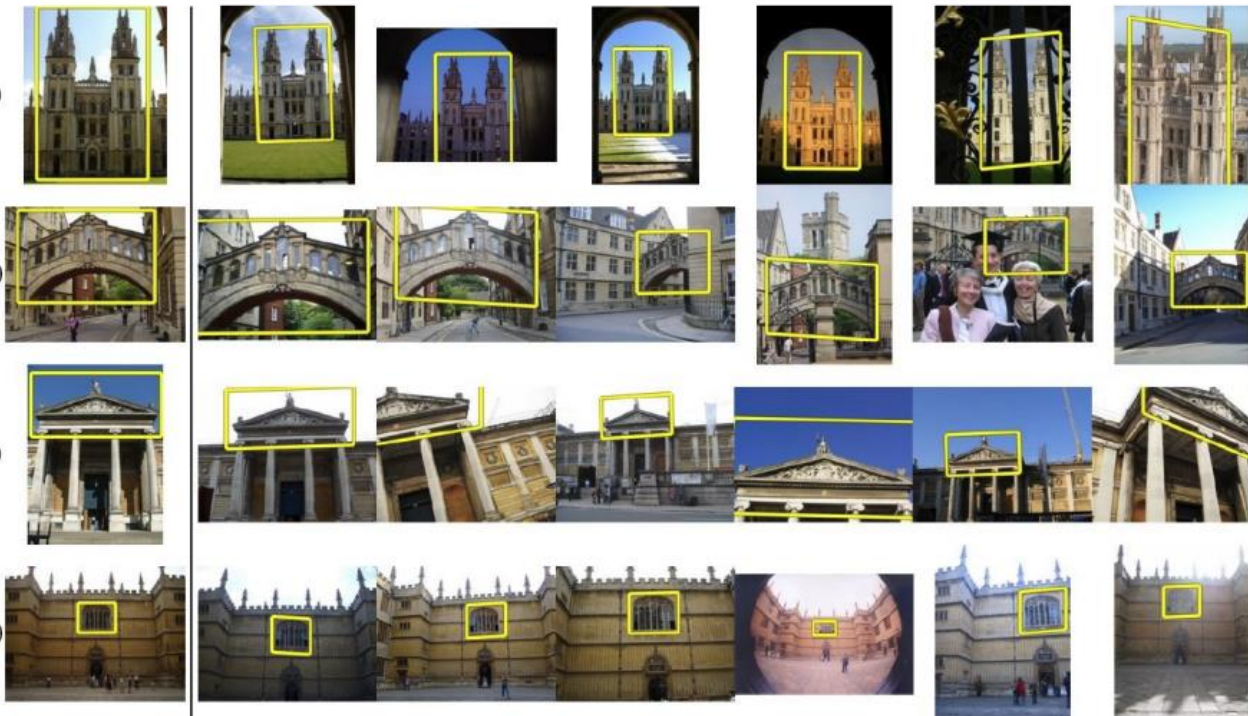


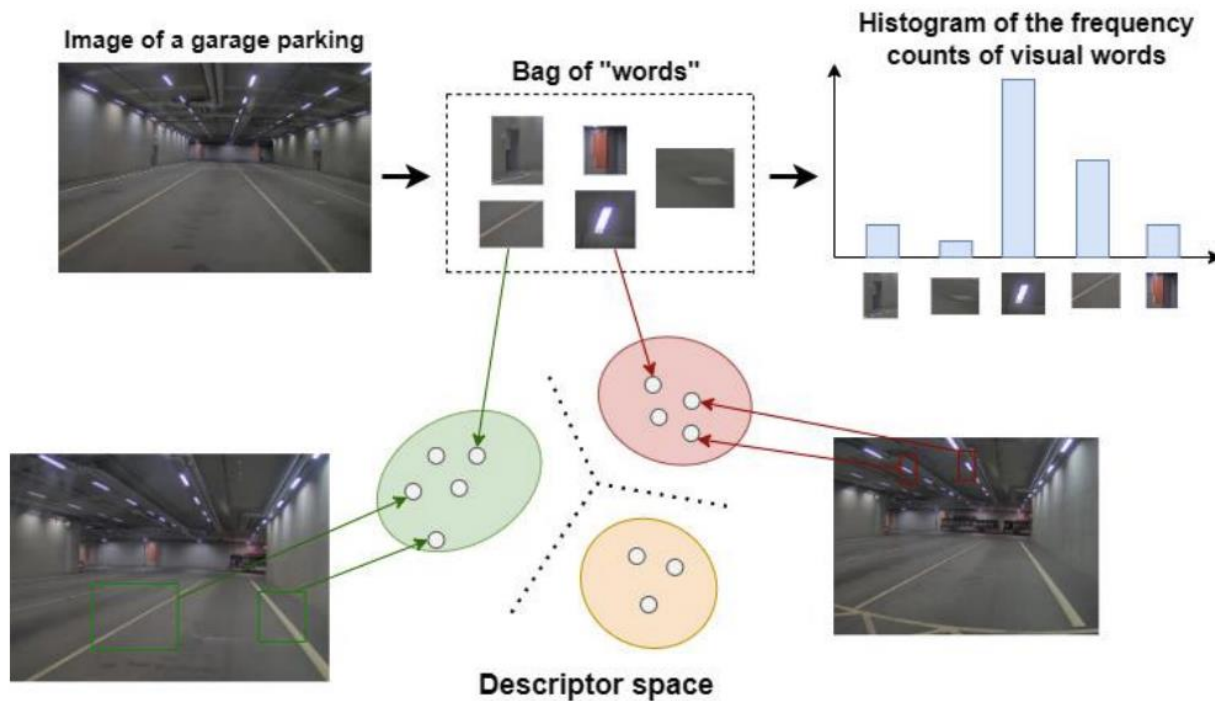
Identify the places that a robot has visited before

EPFL Visual place recognition

Query image

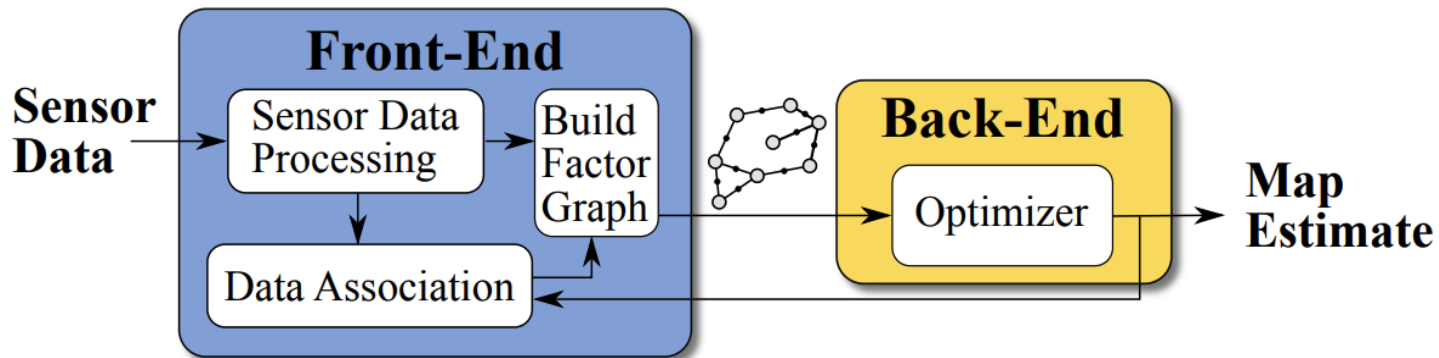
Results on a database of 100 million images



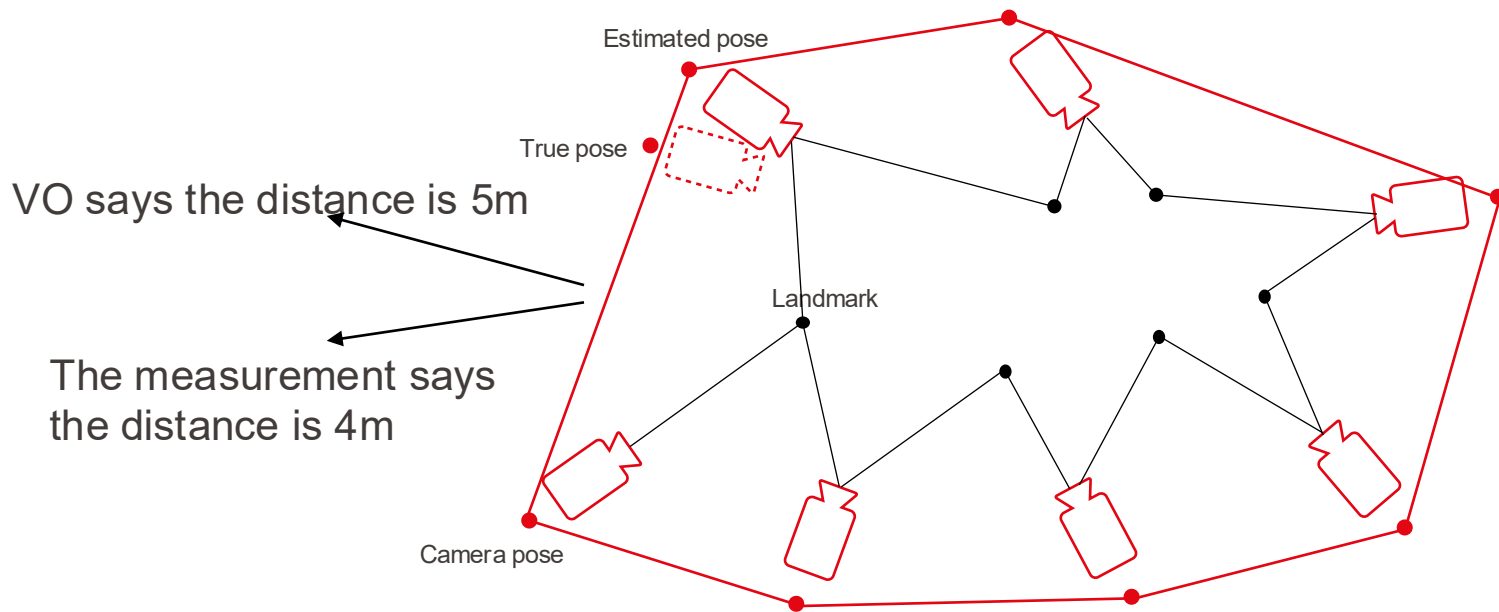


L. Fei-Fei, "Bag of words models", Conference on Computer Vision and Pattern Recognition, 2007

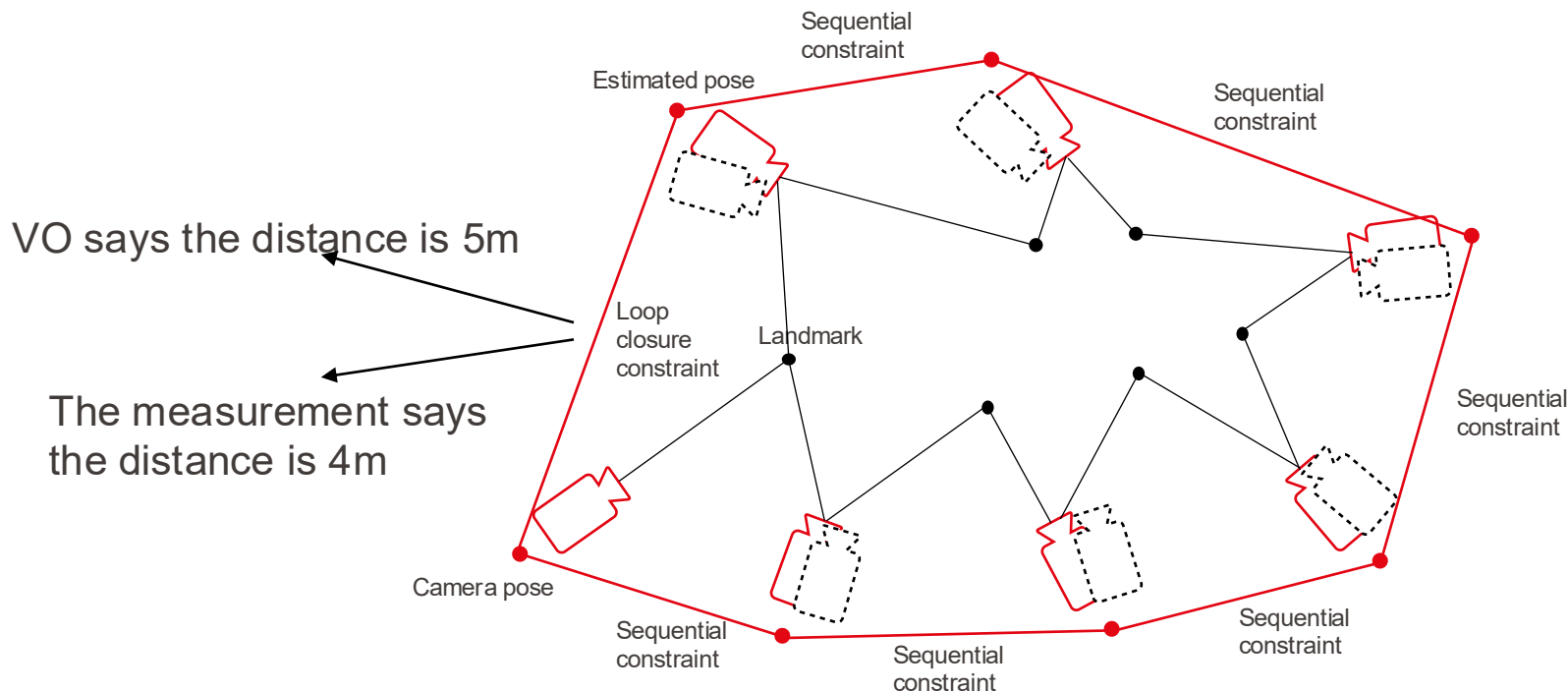
SLAM front-end constructs a pose graph with loop closure edges



- Back-end -> Optimize the pose graph and the map

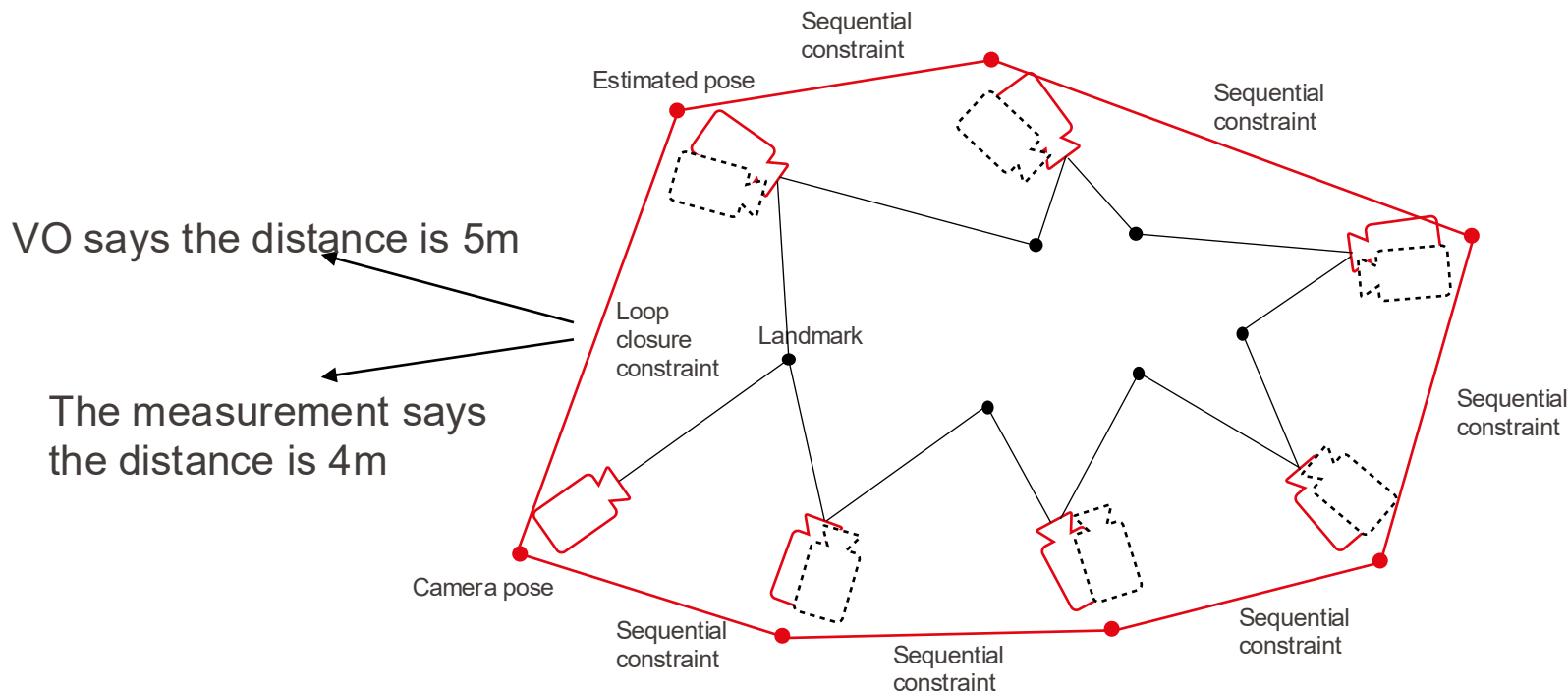


Goal: resolve the inconsistency

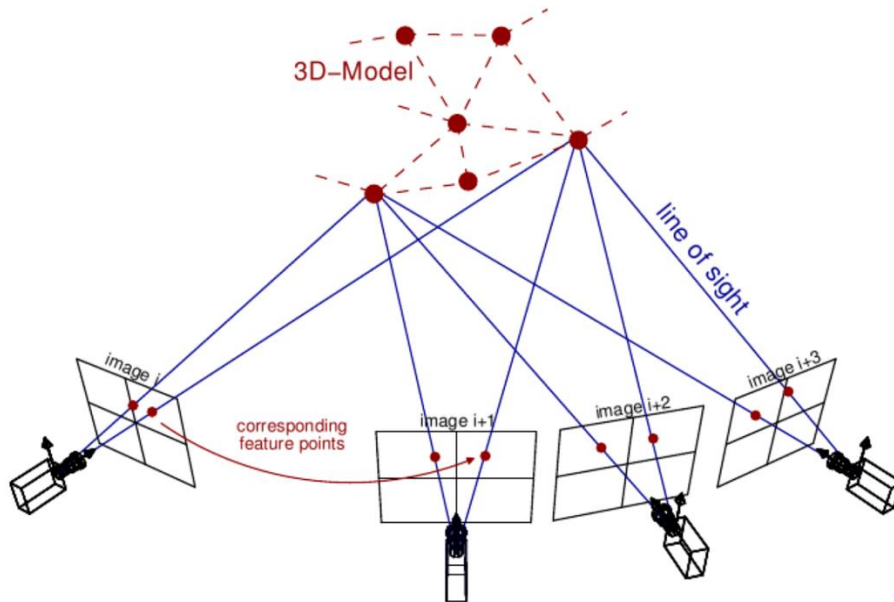


Constraints can be conflicting

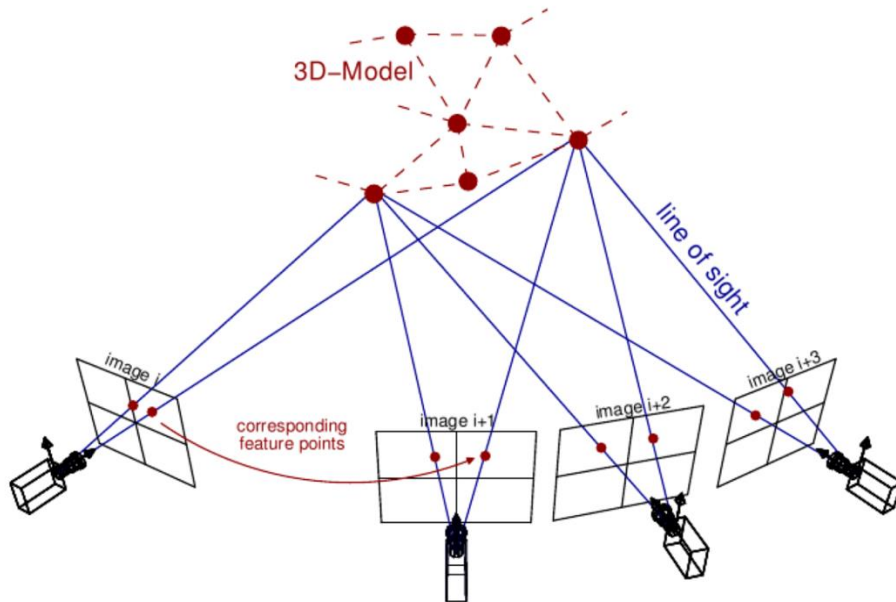
Goal: change the poses to minimize the error introduced by the constraints



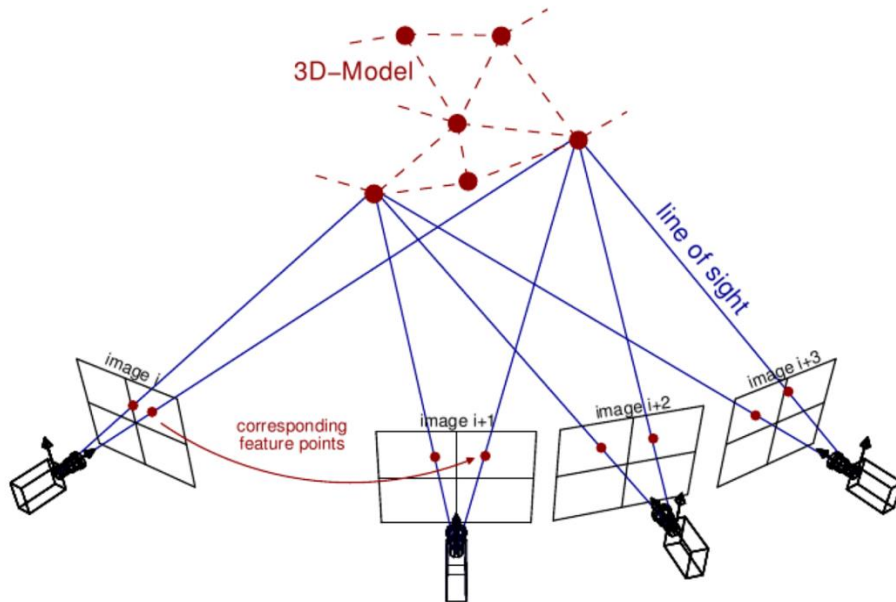
The error is typically the squared difference between the predicted and actual measurement



- A technique developed in 1950s in photogrammetry community
- Now, also widely used in computer vision and robotics

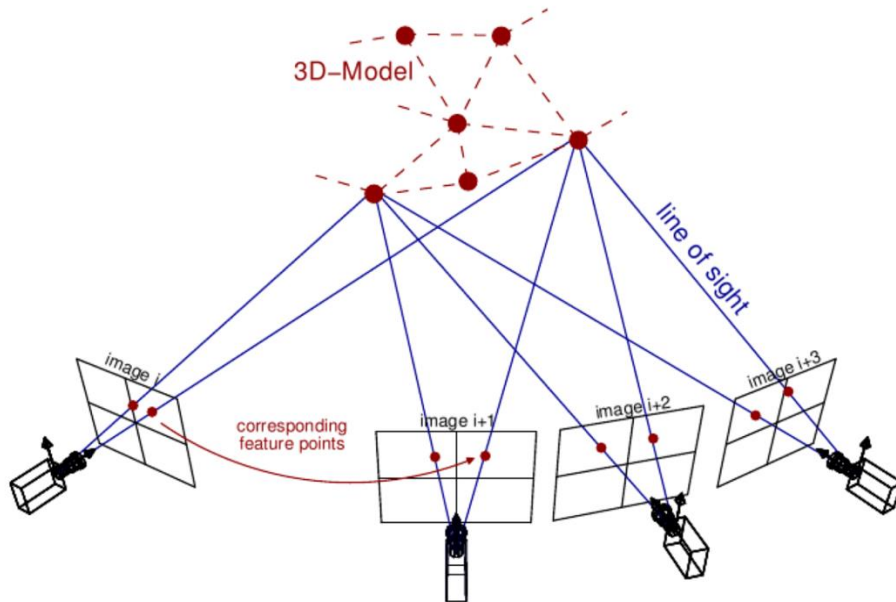


Not only optimize the pose but also the 3D points

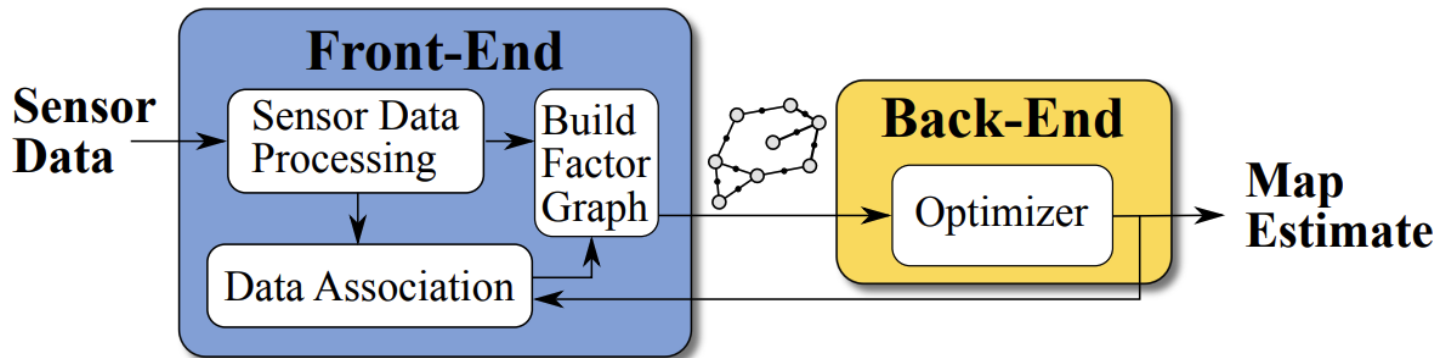


Not only optimize the pose but also the 3D points

- Project the estimated 3D points into estimated camera images
- Compare locations of the projected 3D points with the measured ones (correspondences)
- Adjust to minimize the reprojection errors (in pixels) in images
- Iteratively update the 3D points and camera poses



More precise, but slower



Front-end -> Construct a pose graph

- Correspondences matching
- Outlier detection
- Loop closure detection

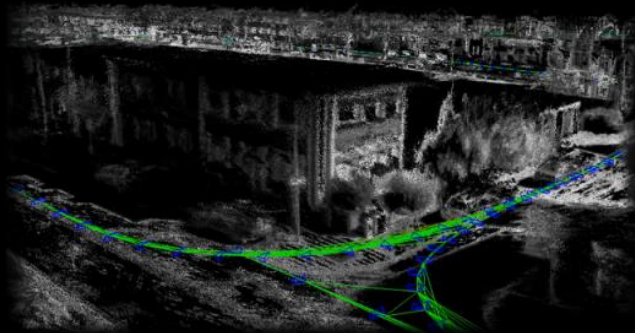
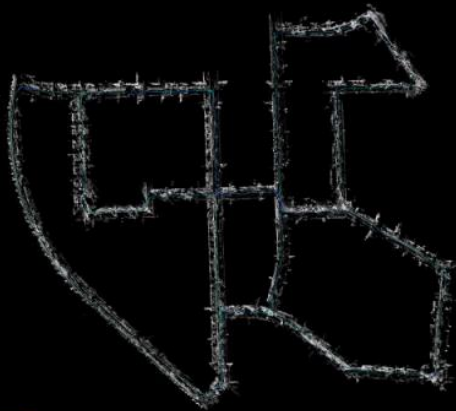
Back-end -> Optimize the pose graph and the map

- Pose graph optimization or bundle adjustment

Let's take another look at LSD-SLAM

Large-Scale Direct SLAM with Stereo Cameras

Jakob Engel, Jörg Stückler, Daniel Cremers
IROS 2015, Hamburg



Computer Vision Group
Technical University Munich



Dense



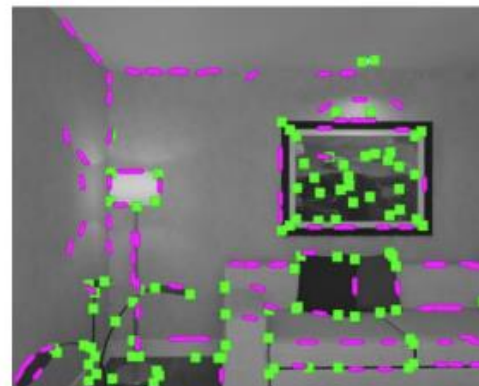
DTAM [Newcombe et al. '11]
300'000+ pixels

Semi-Dense



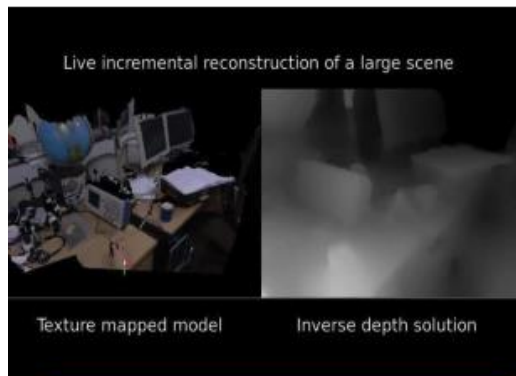
LSD [Engel et al. 2014]
~10'000 pixels

Sparse



SVO [Forster et al. 2014]
100-200 features x 4x4 patch
~ 2,000 pixels

Dense



DTAM [Newcombe et al. '11]
300,000+ pixels

Semi-Dense



LSD-SLAM [Engel et al. 2014]
~10,000 pixels

Sparse



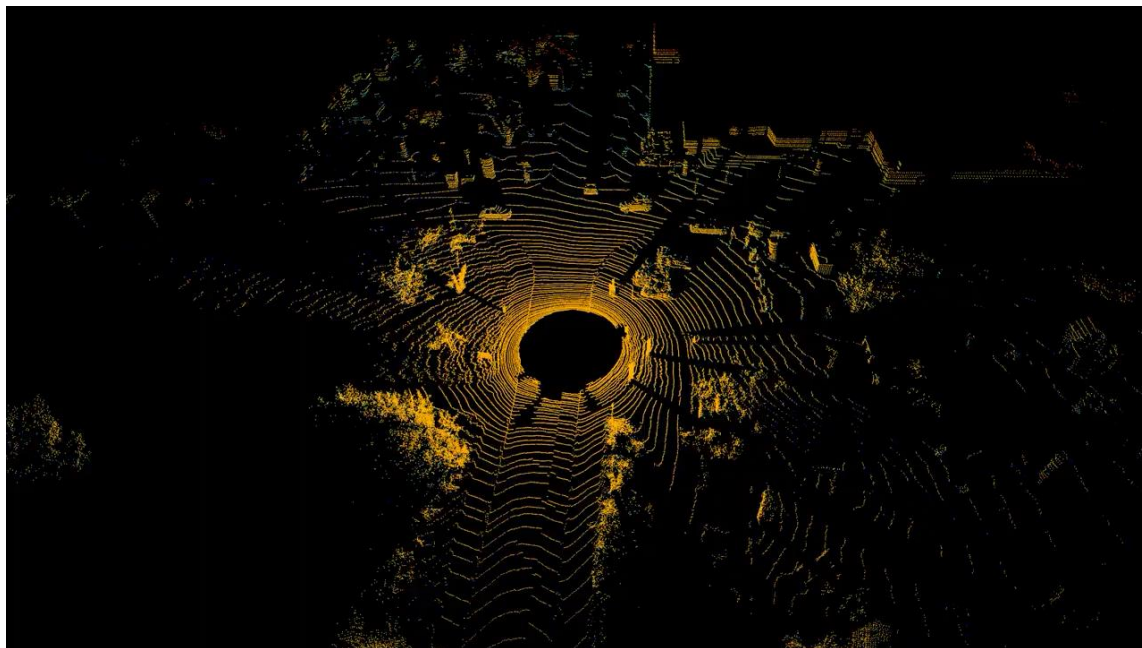
SVO [Forster et al. 2014]
100-200 features x 4x4 patch
~ 2,000 pixels

Vizzo, Ignacio, et al. "Kiss-icp: In defense of point-to-point icp—simple, accurate, and robust registration if done the right way." *IEEE RA-L*.

Visual SLAM:

- Monocular
- Stereo
- RGB-D

LiDAR SLAM

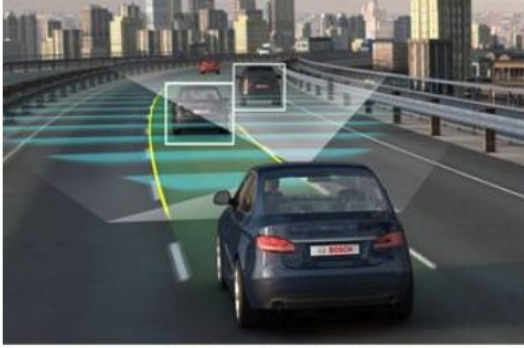


Multi-Sensor SLAM:

- Cameras, IMUs, GPS, LiDARs, Radars

EPFL SLAM applications

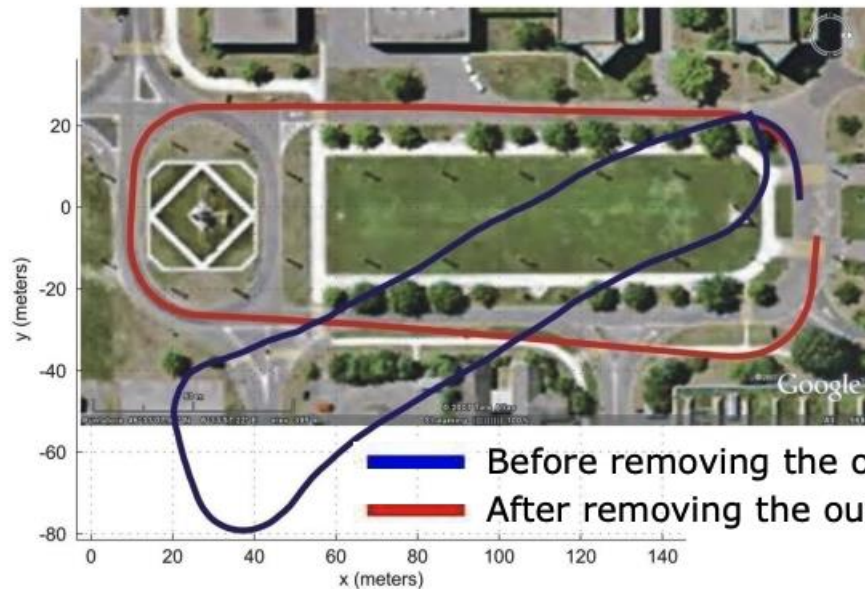
79



Deep Learning for Visual SLAM

Where can deep learning improve SLAM?

What can go wrong in SLAM?



- Error at the loop closure: 6.5 m
- Error in orientation: 5 deg
- Trajectory length: 400 m

Bad correspondences matching results in large errors in VO estimation

Use deep learning to find correspondences

Superpoint

Goal:

- Training a model to extract interest points

Superpoint

Goal:

- Training a model to extract interest points

Interest points:

- 2D locations in an image which are stable and repeatable from different lighting conditions and viewpoints

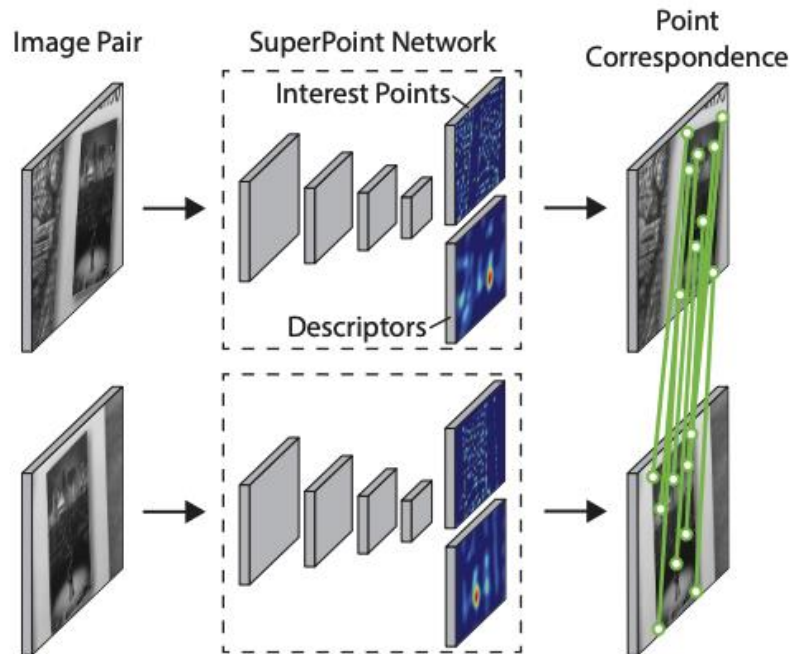
Superpoint

Goal:

- Training a model to extract interest points

Interest points:

- 2D locations in an image which are stable and repeatable from different lighting conditions and viewpoints



Superpoint

Classic method (e.g. SIFT):

- Detect interest points based on manually crafted kernels
- Generate descriptors based on hand engineered features (orientation, illumination)

Superpoint

Classic method (e.g. SIFT):

- Detect interest points based on manually crafted kernels
- Generate descriptors based on hand engineered features (orientation, illumination)

Superpoint:

- Use a convolutional neural network to detect interest points
- Generate descriptors using a network

Superpoint

Classic method (e.g. SIFT):

- Detect interest points based on manually crafted kernels
- Generate descriptors based on hand engineered features (orientation, illumination)

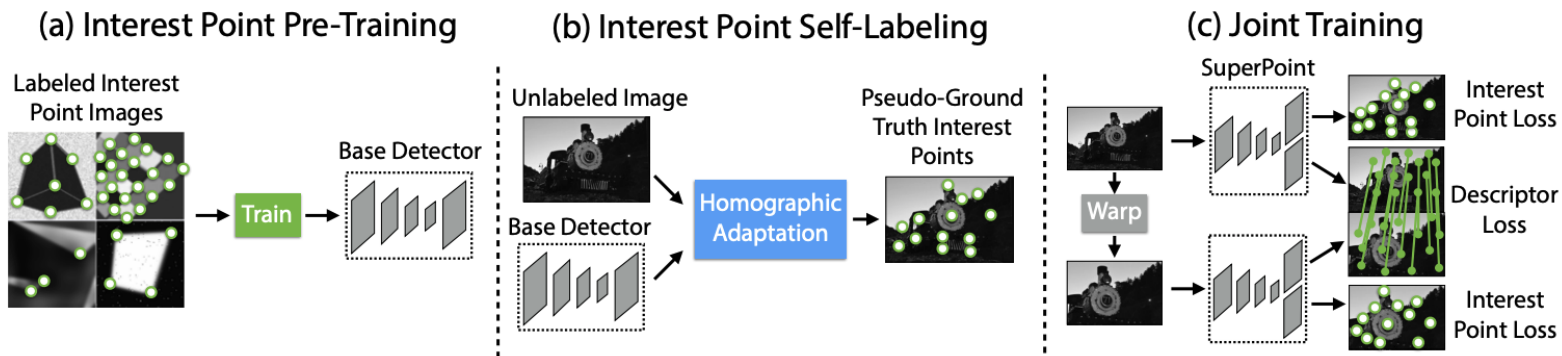
Superpoint:

- Use a convolutional neural network to detect interest points
- Generate descriptors using a network

How to supervise the extraction of interest points?

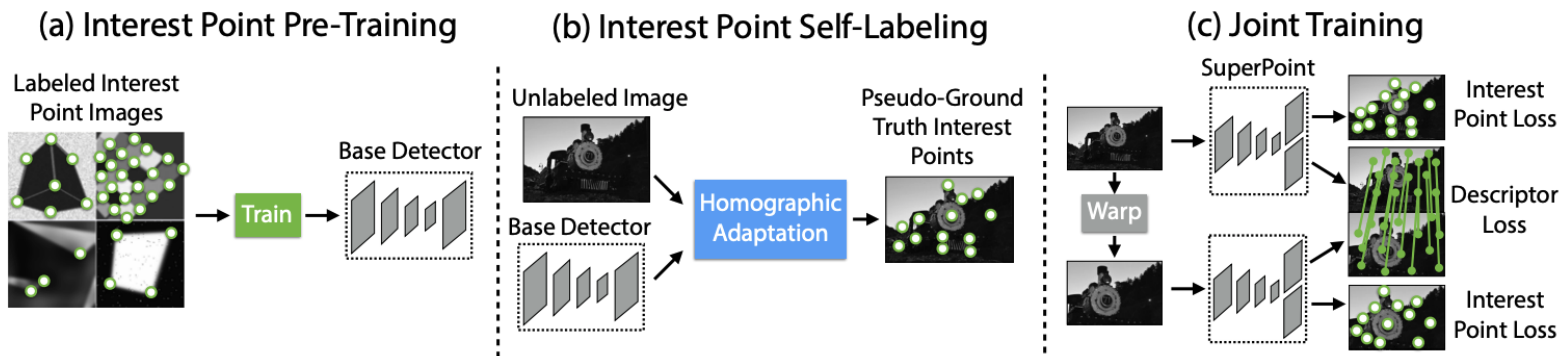
EPFL Deep feature detection and matching

Superpoint

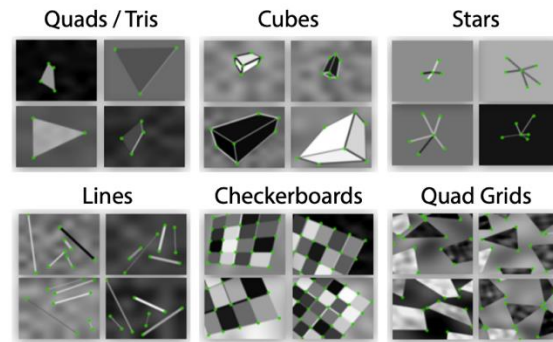


EPFL Deep feature detection and matching

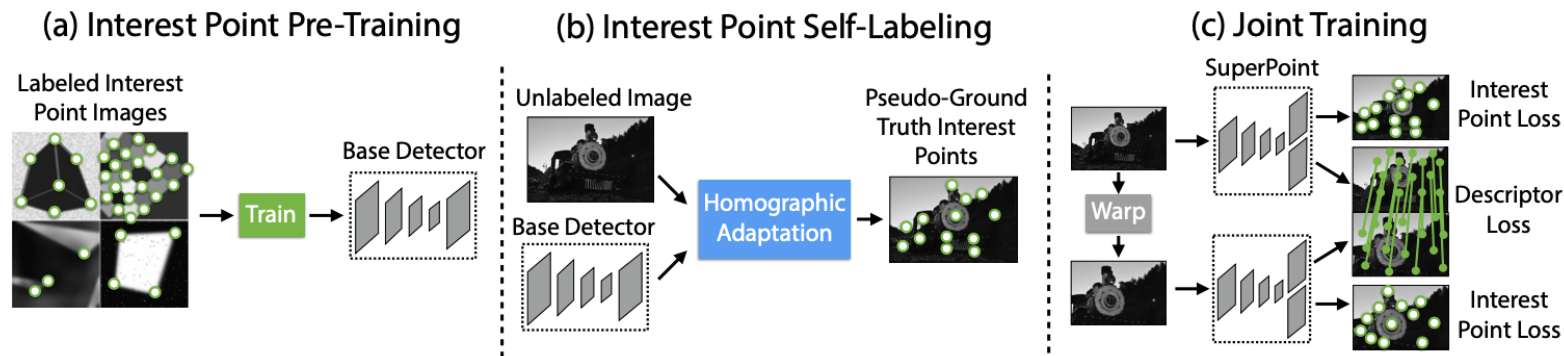
Superpoint



- (a) training a base detector:
- Create a synthetic dataset
 - Supervise the model to detect corners

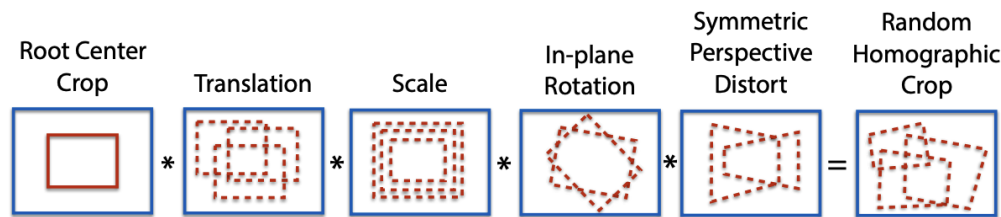


Superpoint



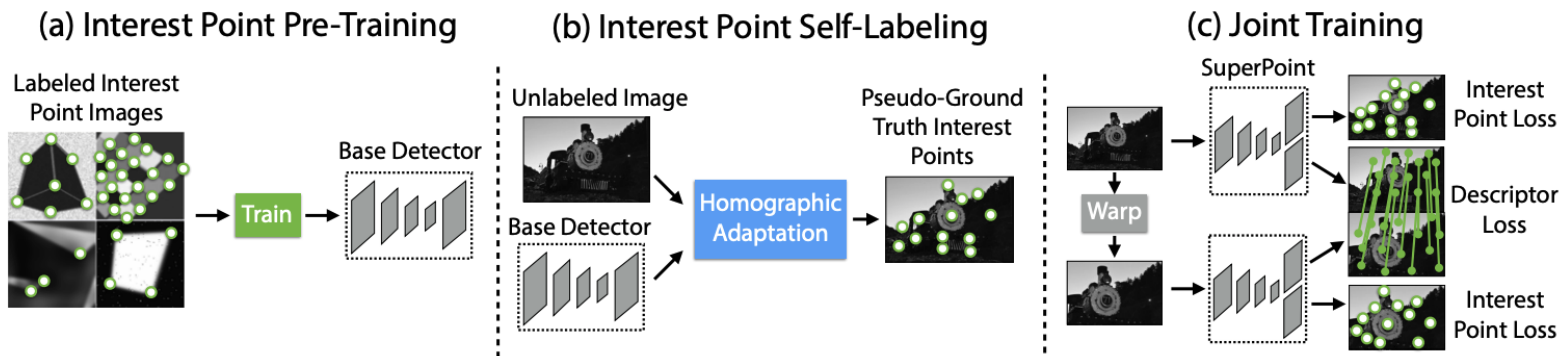
(b) iteratively adapting the detector to real-world image:

- Transform the image with random homography



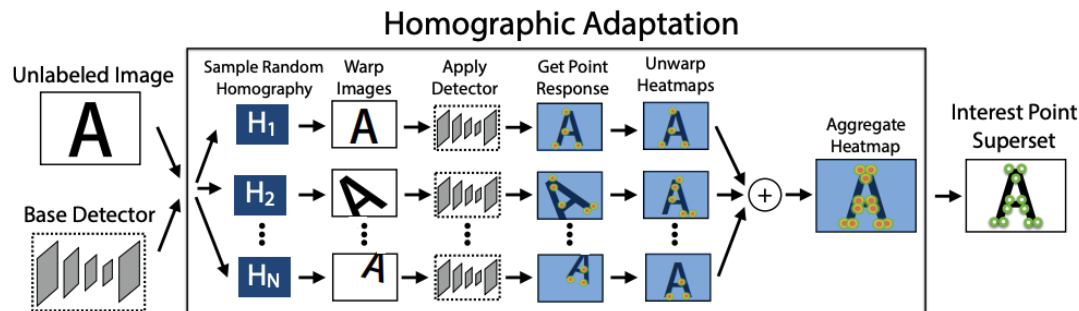
EPFL Deep feature detection and matching

Superpoint



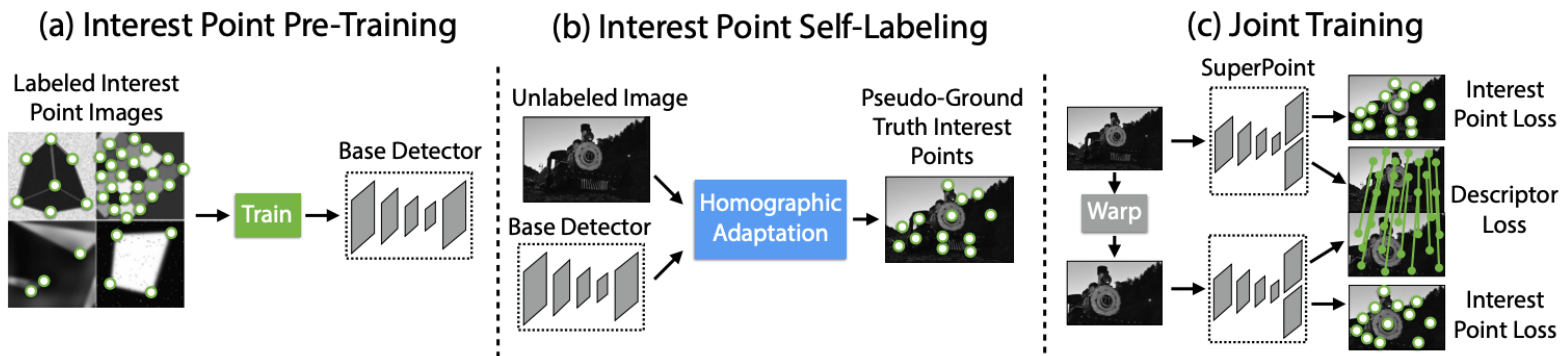
(b) iteratively adapting the detector to real-world image:

- Transform the image with random homography
- Detect on warped images
- Aggregate detection results
- Train base detector



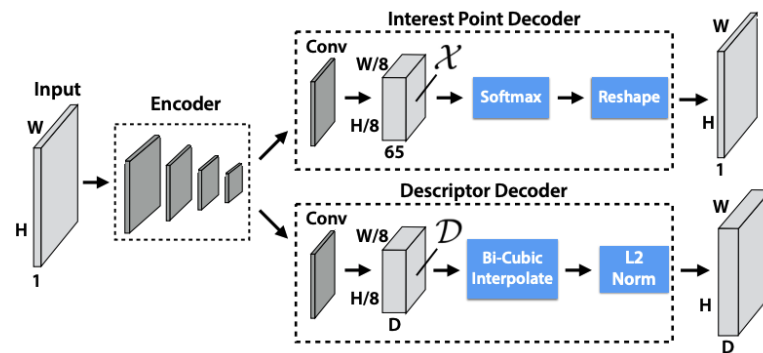
EPFL Deep feature detection and matching

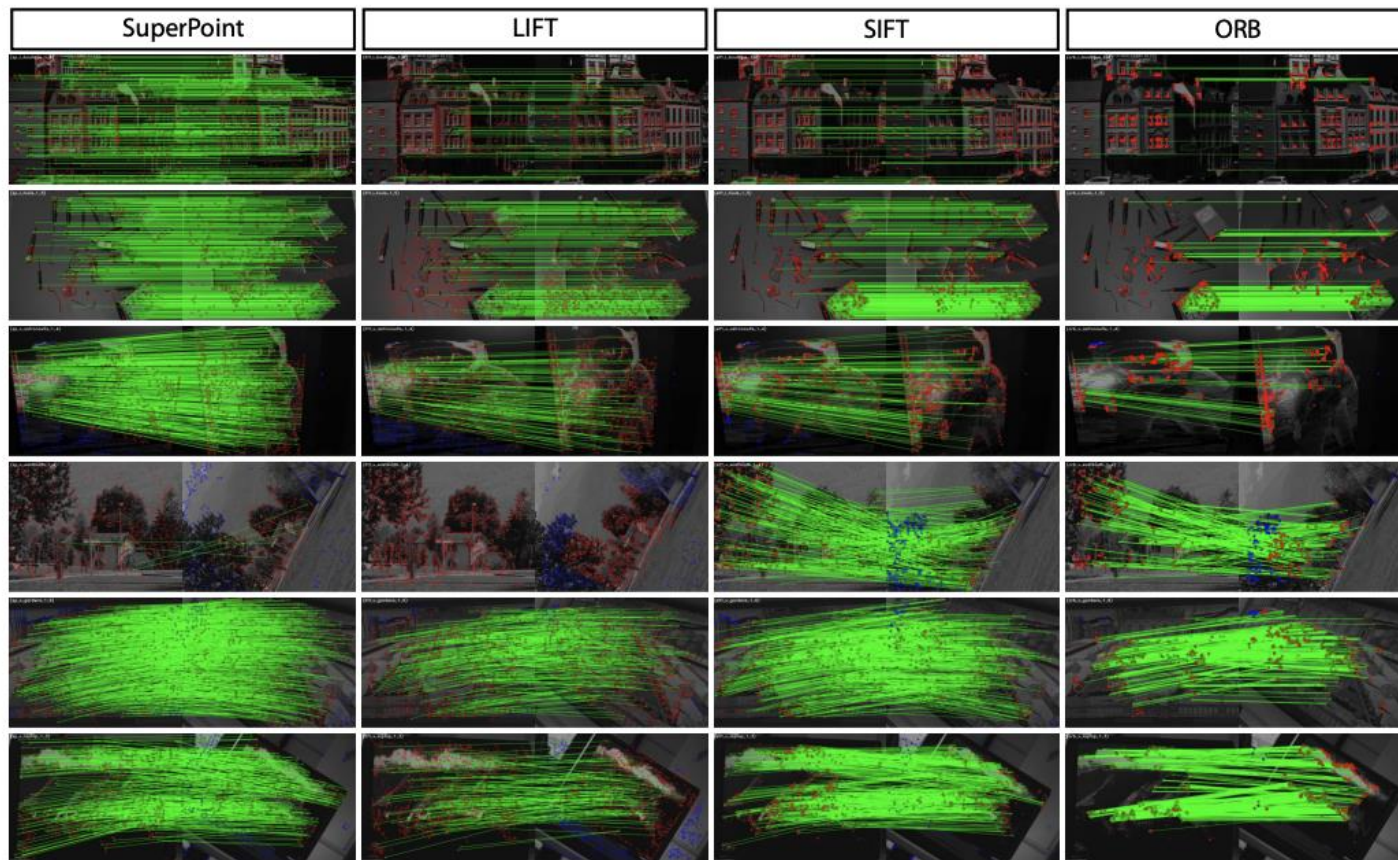
Superpoint



(c) SuperPoint:

- Output a probability map denotes “point-ness”
- Output a dense map of descriptors
- Jointly supervise detection and description





DeTone, Daniel, et al. "Superpoint: Self-supervised interest point detection and description." *CVPR workshops*. 2018.

SuperGlue

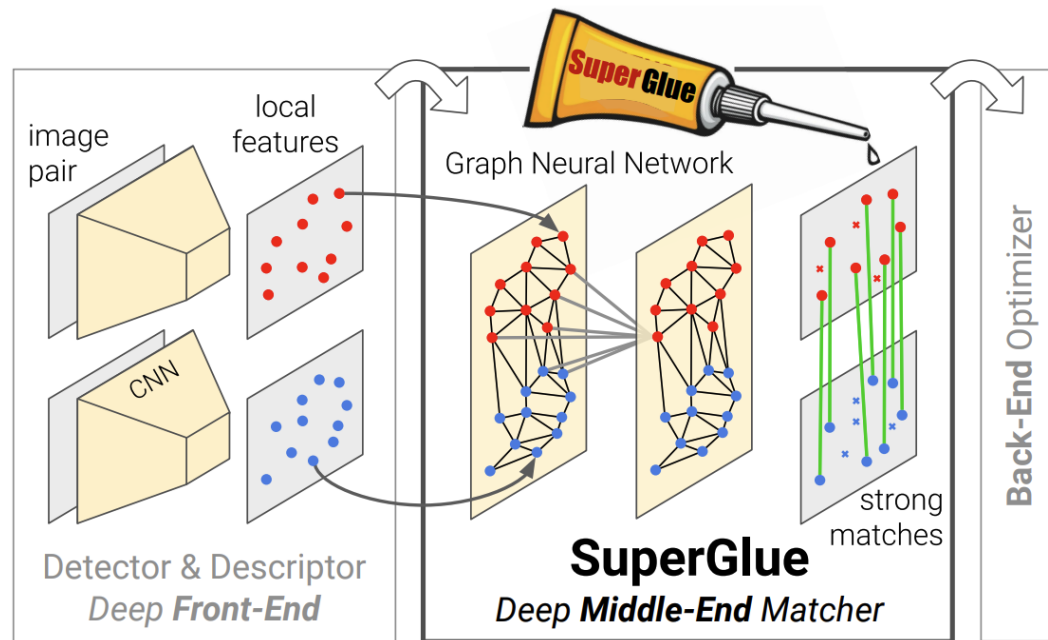
Given two sets of descriptors:

- find correspondences
- reject non-matchable points

SuperGlue

Given two sets of descriptors:

- find correspondences
- reject non-matchable points

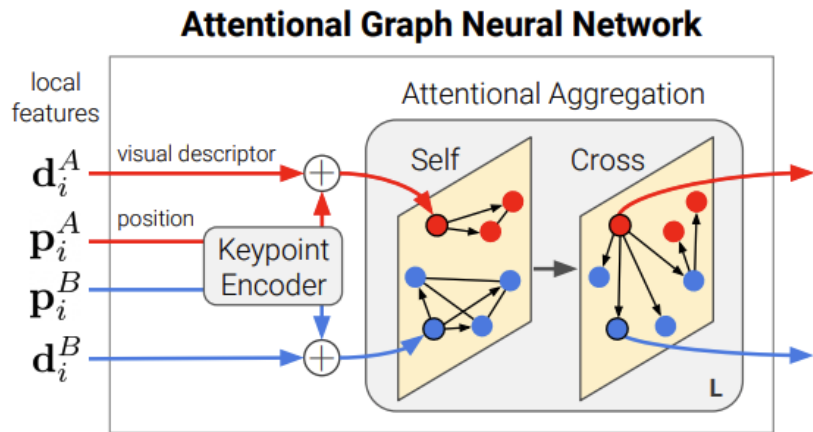


SuperGlue

Motivation:

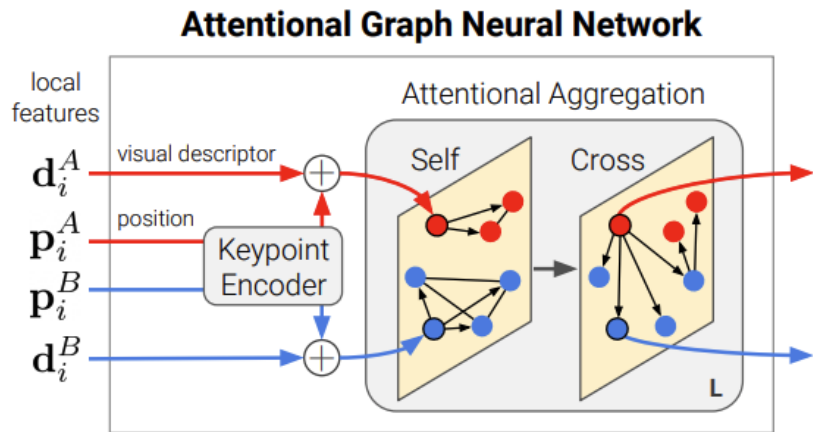
- A keypoint can have at most a single correspondence in the other image
- Some keypoints will be unmatched due to occlusion and failure of the detector

SuperGlue



Input two sets of features (locations + descriptors)

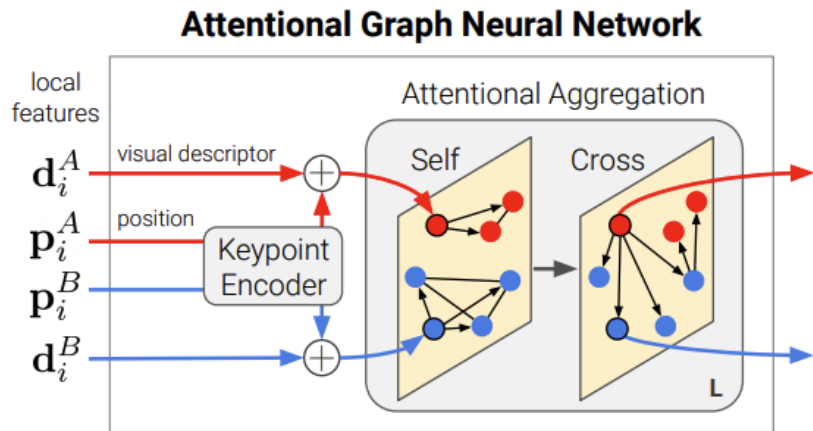
SuperGlue



Input two sets of features (locations + descriptors)

- Construct a complete graph whose nodes are the keypoints

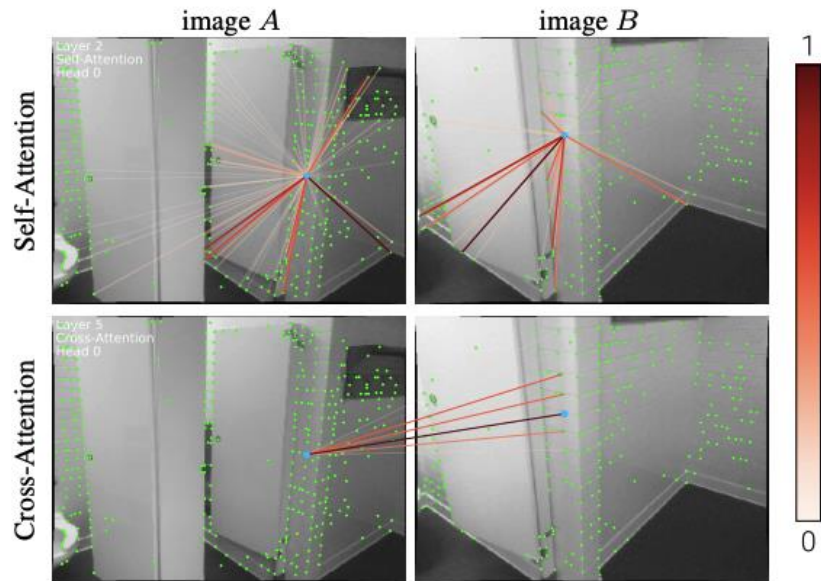
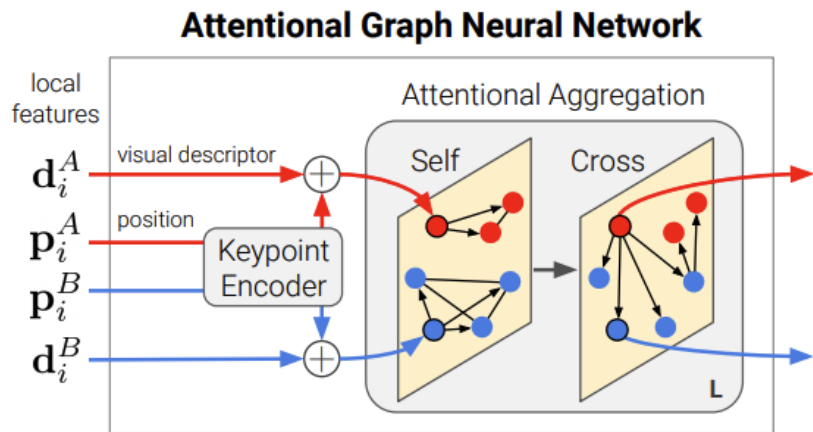
SuperGlue



Input two sets of features (locations + descriptors)

- Construct a complete graph whose nodes are the keypoints
- Mimic humans look back-and-forth at both images

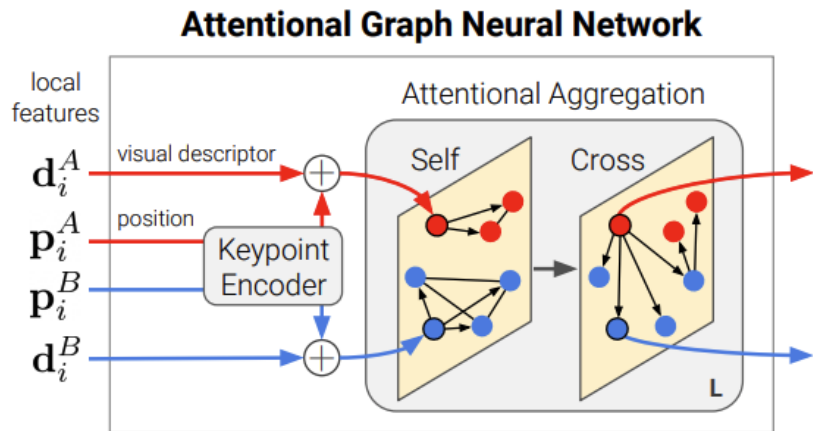
SuperGlue



Input two sets of features (locations + descriptors)

- Construct a complete graph whose nodes are the keypoints
- Mimic humans look back-and-forth at both images

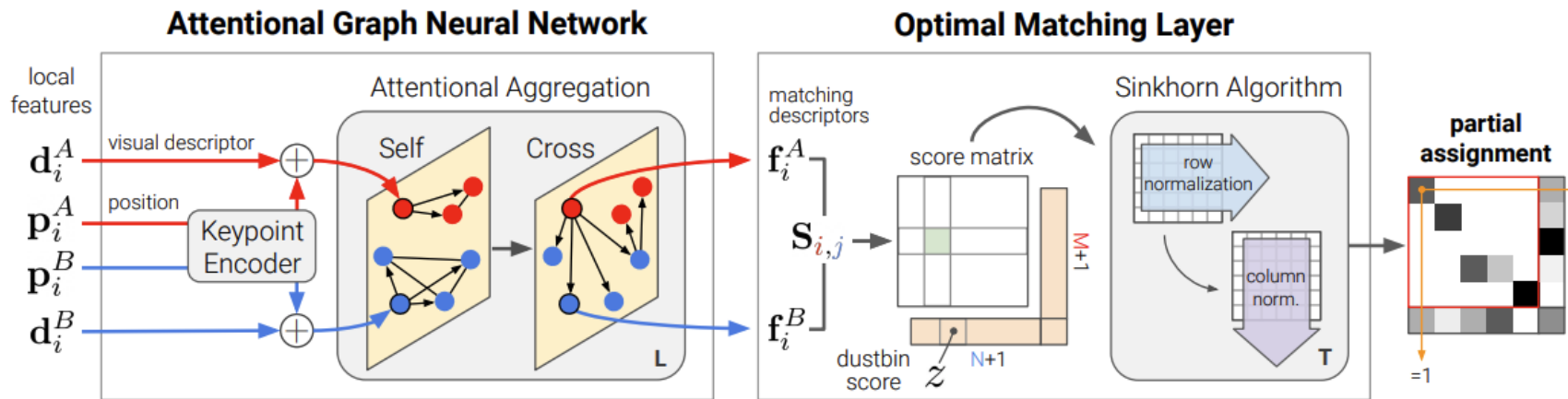
SuperGlue



Input two sets of features (locations + descriptors)

- Construct a complete graph whose nodes are the keypoints
- Mimic humans look back-and-forth at both images
- Output two sets of matching descriptors

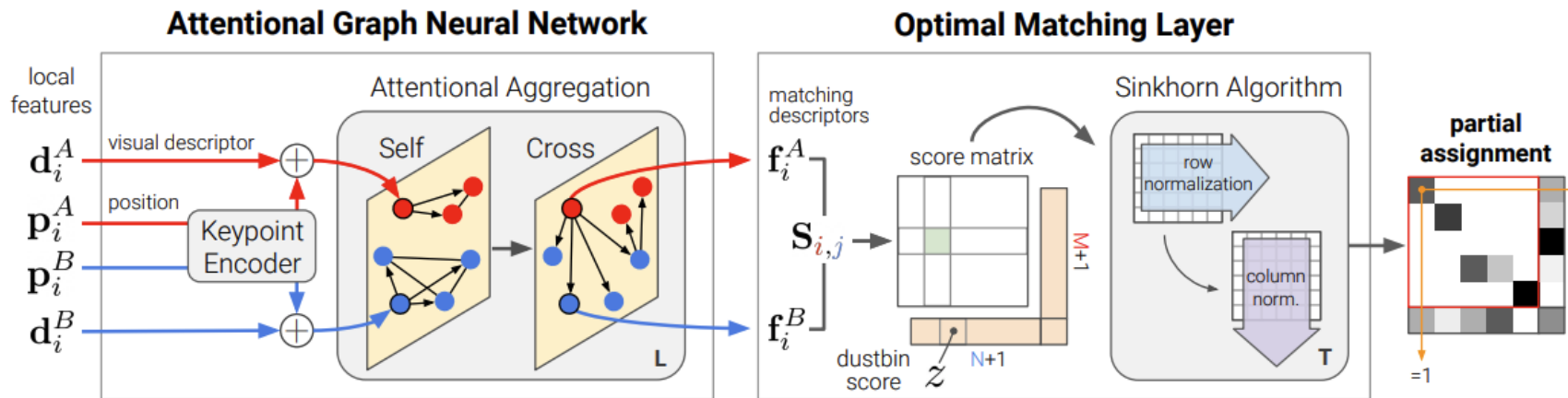
SuperGlue



Input two sets of features (locations + descriptors)

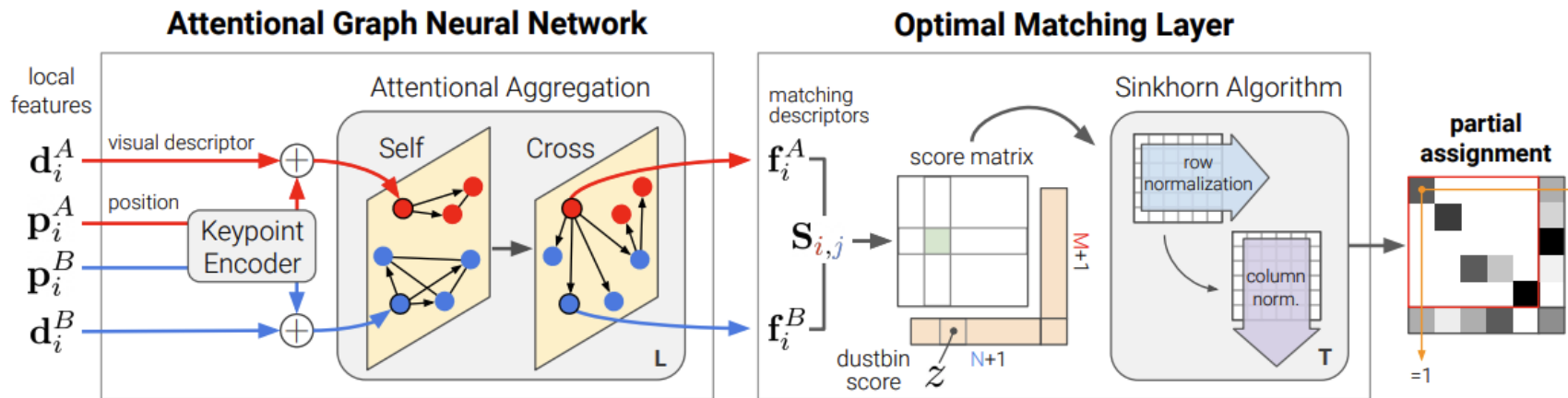
- Construct a complete graph whose nodes are the keypoints
- Mimic humans look back-and-forth at both images
- Output two sets of matching descriptors

SuperGlue



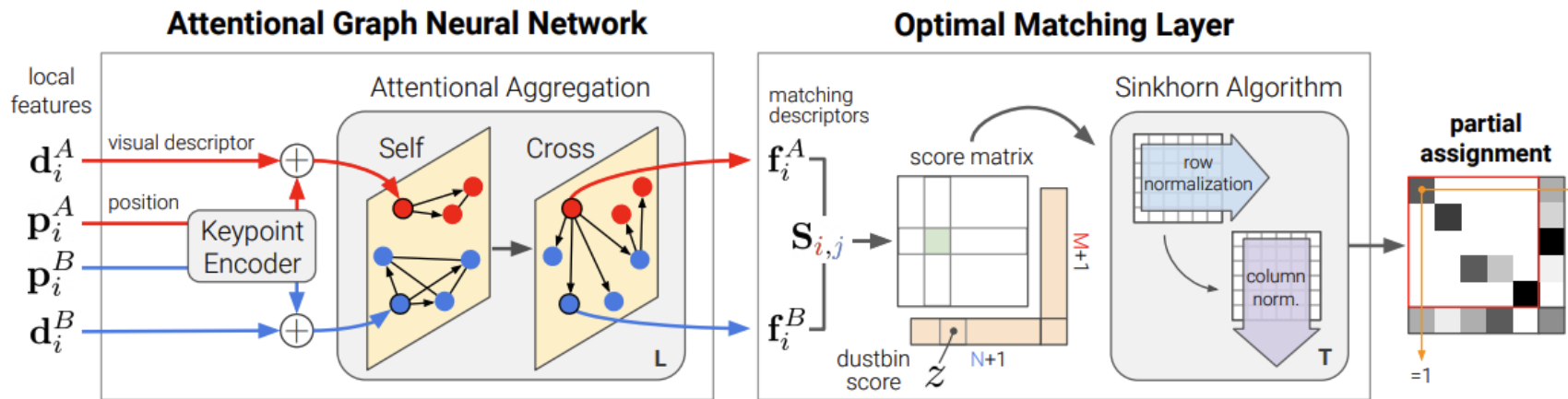
- Compute pairwise similarity score

SuperGlue



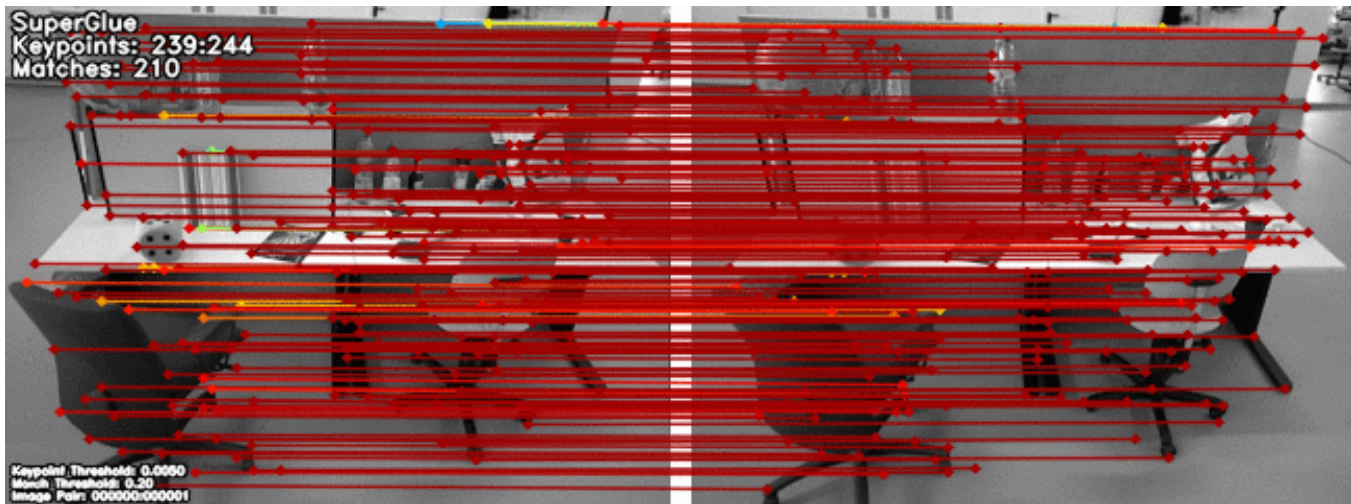
- Compute pairwise similarity score
- Appending a learnable dustbin to account unmatched points

SuperGlue



- Compute pairwise similarity score
- Appending a learnable dustbin to account unmatched points
- Use a differentiable Sinkhorn algorithm to normalize the sum of rows and columns of the score matrix to 1

SuperGlue

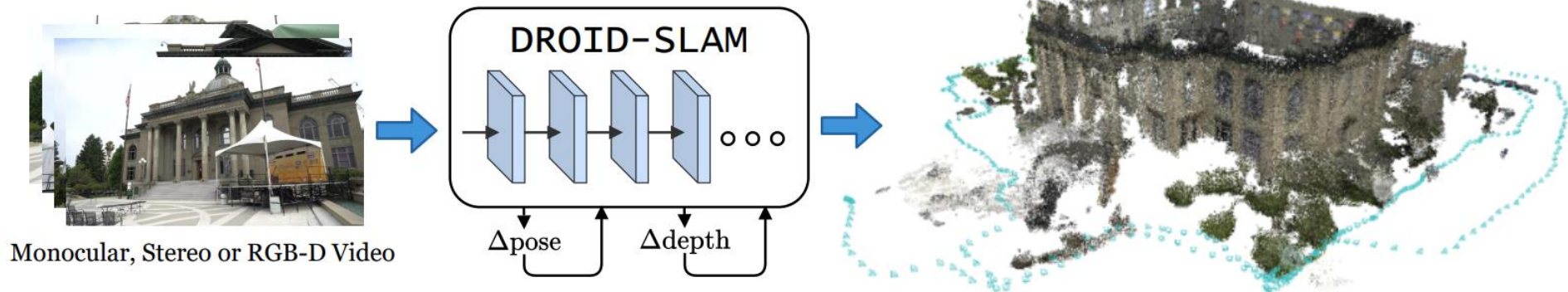


- Able to handle partial assignments and occluded points

Can we use deep learning for end-to-end SLAM?

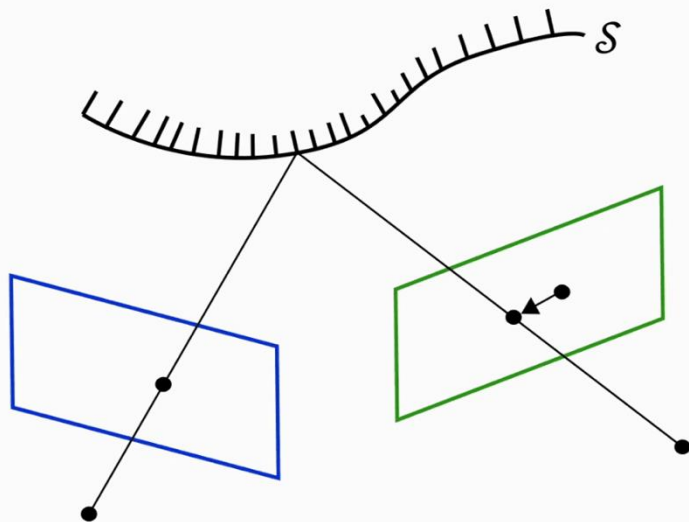
DROID-SLAM

DROID: Differentiable Recurrent Optimization-Inspired Design



DROID-SLAM

Optical flow is an analytical function of depth and camera poses

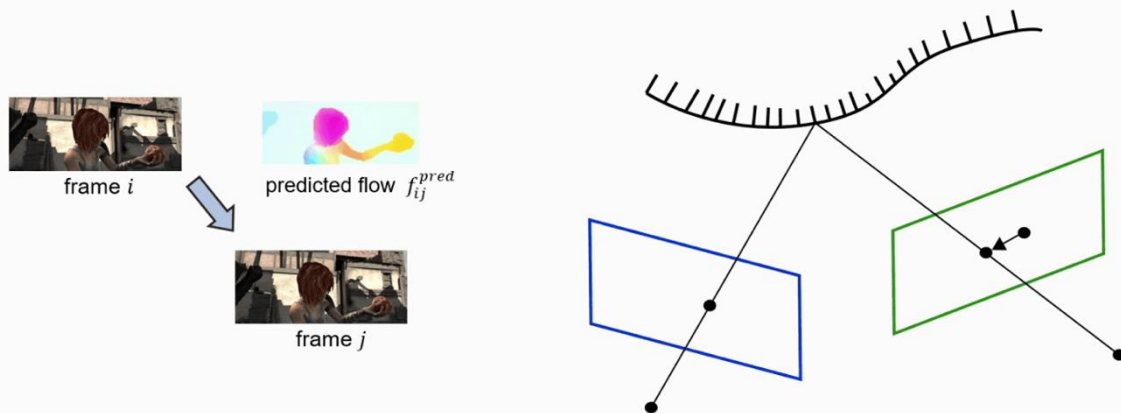


Teed, Zachary, and Jia Deng. "Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras." NeurIPS, 2021

DROID-SLAM

Optical Flow to Depth & Camera Poses

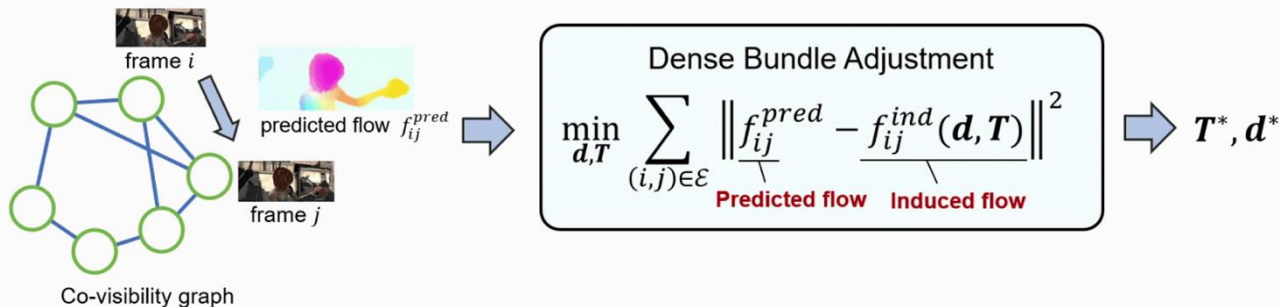
Dense Bundle Adjustment (DBA): find pixelwise depth and poses consistent with predicted flow



DROID-SLAM

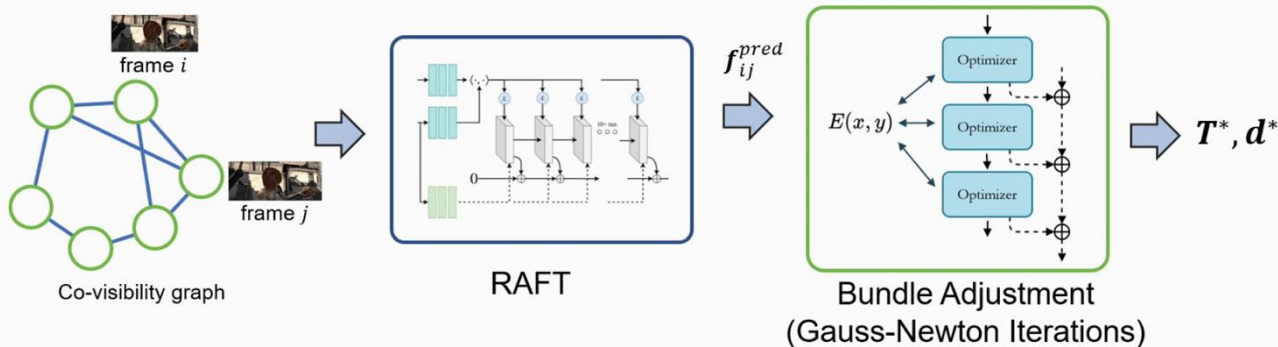
Dense Bundle Adjustment

- **Given:** co-visibility graph $(\mathcal{V}, \mathcal{E})$, predicted flow f_{ij}^{pred}
- **Want:** depth maps $\mathbf{d} = (d_1, \dots, d_i, \dots)$, camera poses $\mathbf{T} = (T_1, \dots, T_i, \dots)$



DROID-SLAM

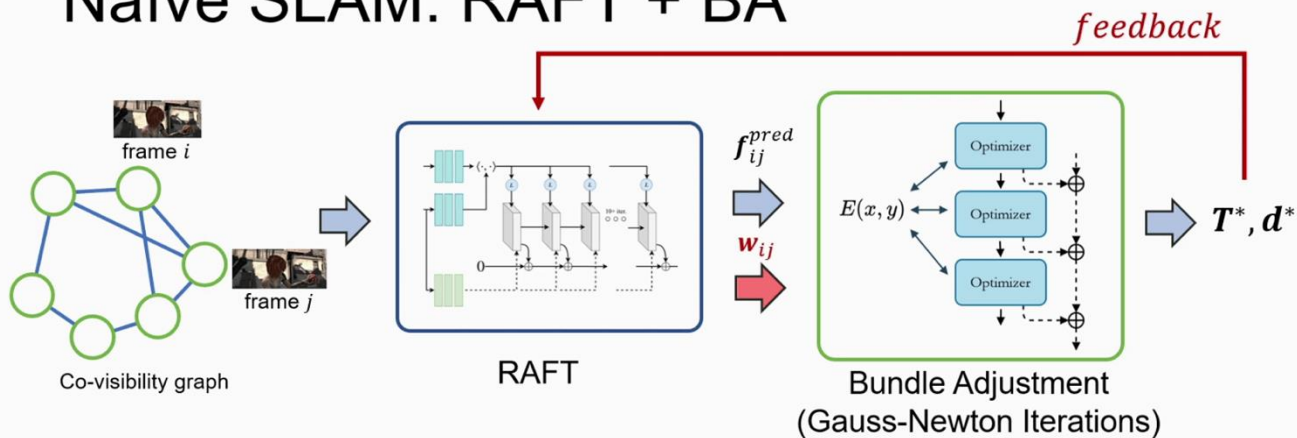
Naïve SLAM: RAFT + BA



- Works poorly, because of outliers: visibility, dynamic objects, prediction error

DROID-SLAM

Naïve SLAM: RAFT + BA



- Works poorly, because of outliers: visibility, dynamic objects, prediction error
- How to exclude outliers? (1) Predicted Confidence Map (2) Feedback

DROID-SLAM

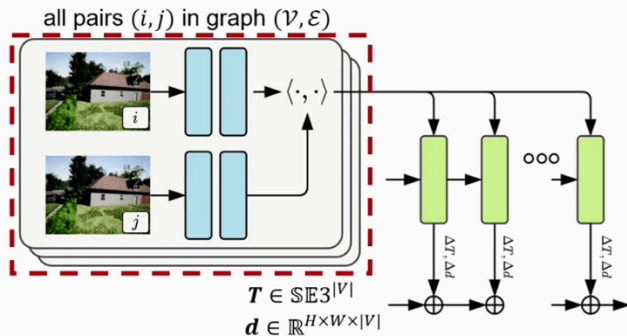
DROID-SLAM: Architecture

- Iterative optimization of all depths and poses
- Analytically constrained updates

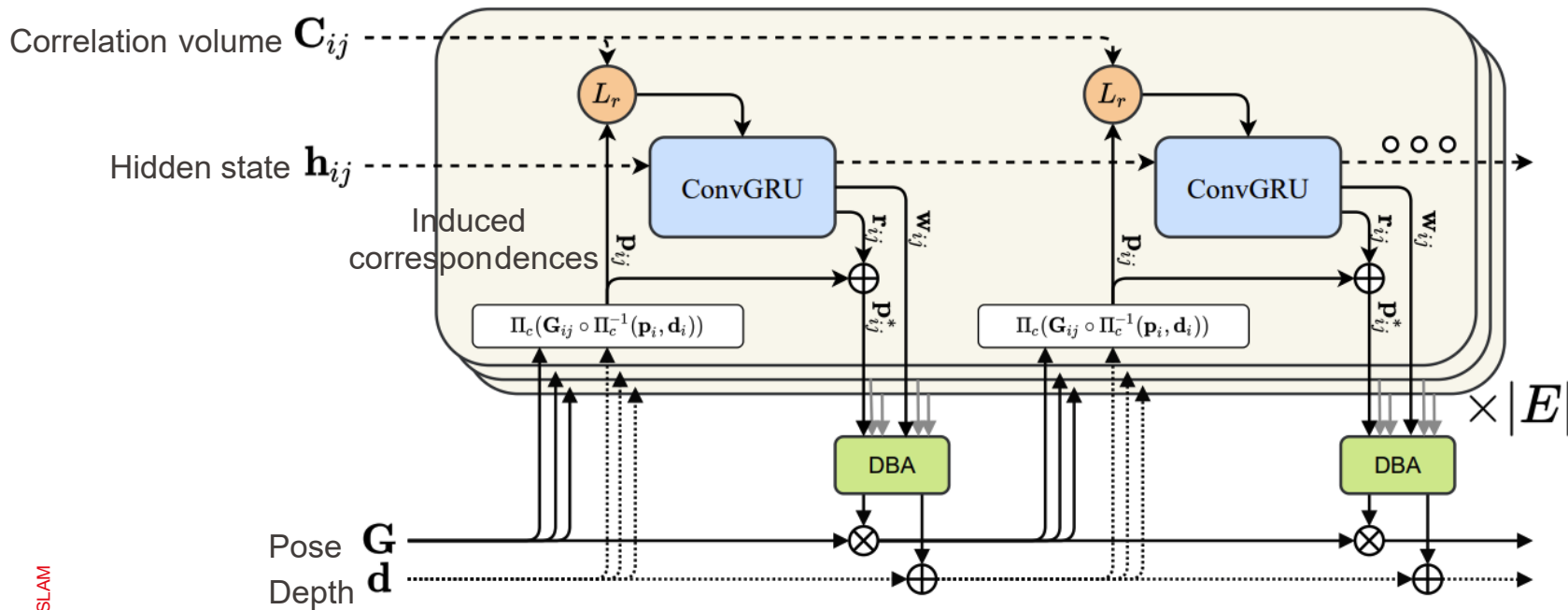
DROID-SLAM

DROID-SLAM: Architecture

- Iterative optimization of all depths and poses
- Analytically constrained updates

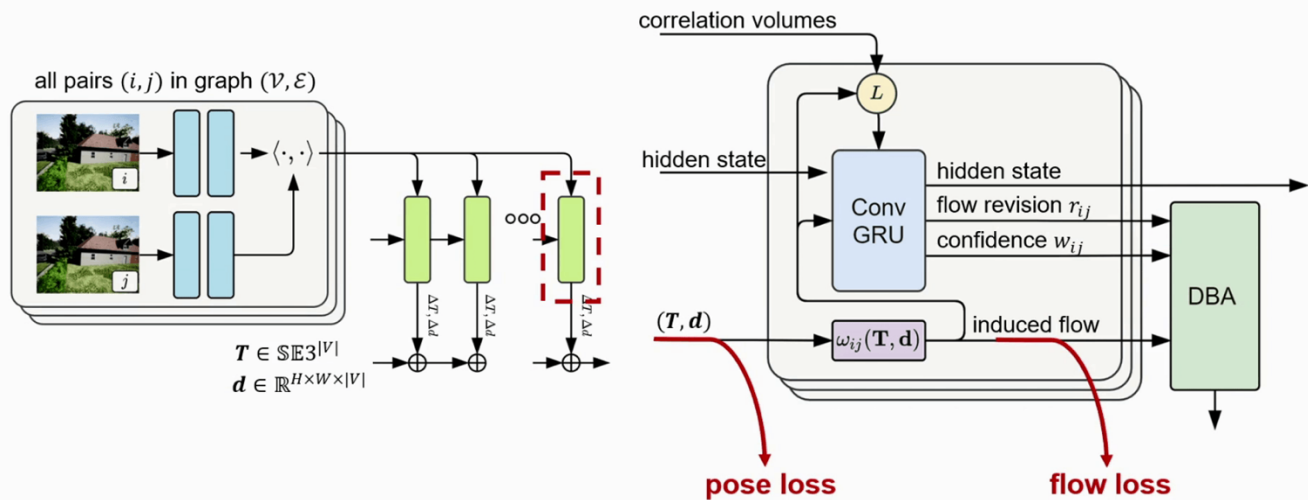


DROID-SLAM



DROID-SLAM

Supervision



DROID-SLAM

In inference, the system contains two threads which run asynchronously

Front-end:

- Takes in new frames, extracts features, selects key frames, perform local bundle adjustment

Back-end:

- Performs global bundle adjustment over the entire history of keyframes

Teed, Zachary, and Jia Deng. "Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras." NeurIPS, 2021

DROID-SLAM

Deep Visual SLAM for Monocular,
Stereo, and RGB-D Cameras

Zachary Teed and Jia Deng
Princeton University



SLAM front-end

- Feature extraction, matching, and outlier removal
- Loop closure detection

SLAM front-end

- Feature extraction, matching, and outlier removal
- Loop closure detection

SLAM back-end

- Optimize the pose graph and the map

SLAM front-end

- Feature extraction, matching, and outlier removal
- Loop closure detection

SLAM back-end

- Optimize the pose graph and the map

Deep learning for:

- Correspondences matching
- End-to-end SLAM